A Project Report

On

# "SMART CITY ISSUE REPORTING AND MANAGEMENT SYSTEM"

Submitted to the

**Department of Computer Applications**

In partial fulfilment of the Course

**Integrated Master of Computer Applications**

Under the guidance of

**Mr. MARIADAS RONNIE C P**


**Project Done by**

NITHYA JOY

(Reg no: 213242110120)




**DEPARTMENT OF COMPUTER APPLICATIONS**

**SCMS SCHOOL OF TECHNOLOGY AND MANAGEMENT**


***May-2026***

# SCMS SCHOOL OF TECHNOLOGY AND MANAGEMENT

## BONAFIDE CERTIFICATE

Certified that the Project Work entitled

## "Smart City Issue Reporting and Management System"

**is a Bonafide work done by**

**Nithya Joy**

*In partial fulfillment of the requirement for the Award of*

## INTEGRATED MASTER OF COMPUTER APPLICATIONS

**Degree From**
Mahatma Gandhi University, Kottayam
(2021-2026)

*Head of Department* *Project Guide*

*Submitted for the Viva-Voce Examination held on……..*

**External Examiner1**

(Name & Signature)

**External Examiner2**

(Name & Signature)

# SCMS SCHOOL OF TECHNOLOGY AND MANAGEMENT


## CERTIFICATE

This is to certify that the project entitled **"SMART CITY ISSUE REPORTING AND MANAGEMENT SYSTEM"** has been successfully carried out by **NITHYA JOY**(Reg no: 213242110120) in partial fulfilment of the Course **INTEGRATED MASTER OF COMPUTER APPLICATIONS.**

Date: **HEAD OF DEPARTMENT**

# SCMS SCHOOL OF TECHNOLOGY AND MANAGEMENT

## CERTIFICATE

This is to certify that the project entitled **"SMART CITY ISSUE REPORTING AND MANAGEMENT SYSTEM"** has been successfully carried out by **NITHYA JOY**(Reg no: 213242110120) in partial fulfilment of the Course **INTEGRATED MASTER OF COMPUTER APPLICATIONS.**

Date:

Mr. Mariadas Ronnie C P
INTERNAL GUIDE

# SCMS SCHOOL OF TECHNOLOGY AND MANAGEMENT


## DECLARATION

I, **NITHYA JOY**, hereby declare that the project work entitled **"SMART CITY ISSUE REPORTING AND MANAGEMENT SYSTEM "** is an authenticated work carried out by me under the guidance of **Mr. Mariadas Ronnie C P** for the partial fulfilment of the course **INTEGRATED MASTER OF COMPUTER APPLICATIONS**. This work has not been submitted for similar purpose anywhere else except to **SCMS SCHOOL OF TECHNOLOGY AND MANAGEMENT**, affiliated to **M.G. UNIVERSITY, KOTTAYAM**.

I understand that detection of any such copying is liable to be punished in any way the school deems fit.

**Date:**

**Place:** **NITHYA JOY**

# ACKNOWLEDGEMENT

An endeavor over a long period can be successful with the advice, support, and blessings of many well-wishers. To acknowledge all of them is a blissful opportunity showered upon me by the Almighty. With great pleasure and privilege, I present here with full satisfaction, the Project report on **" SMART CITY ISSUE REPORTING AND MANAGEMENT SYSTEM ".** I take this opportunity to express my gratitude and sincere thanks to all who helped me complete this project successfully.

I first thank God Almighty, who showered His immense blessings on my effort. I express my gratitude and sincere thanks to the Registrar and Group Director**, Dr. Indu Nair**, for her kind consideration and valuable guidelines throughout our project work. Special thanks to **Dr. J.M. Lakshmi Mahesh**, our Principal, for her unwavering support and encouragement.

I sincerely express my gratitude to **Dr. Jismy Joseph**, Program Coordinator, Department of Computer Applications, SCMS School of Technology and Management (SSTM). I extend my sincere gratitude to **Mr. Mariadas Ronnie C P**, my Project Guide, for his valuable guidance, cooperation, and constant encouragement throughout the project work.

I also thank my friends and well-wishers who have provided their wholehearted support to me in this exercise. I believe that this endeavor has prepared me for taking up new challenging opportunities in the future.

**NITHYA JOY**

# Table of Contents

# 1. EXECUTIVE SUMMARY

The **Smart City Issue Reporting & Management System** is a comprehensive full-stack web application developed using the Django framework to modernize and streamline the process of reporting and resolving civic infrastructure issues. The primary purpose of this system is to provide citizens with a structured, transparent, and efficient digital platform to report issues such as potholes, garbage accumulation, water leakage, streetlight failures, and sewage problems, while enabling municipal authorities to manage, track, and resolve these complaints effectively. In traditional systems, complaints are often reported through informal channels such as phone calls, in-person visits, or social media platforms, leading to a lack of accountability, duplication of complaints, delayed responses, and absence of tracking mechanisms. These limitations result in inefficiency and poor service delivery.

The proposed system addresses these challenges by introducing an integrated, role-based workflow supported by automation and Artificial Intelligence. Citizens can submit complaints with geolocation and photographic evidence, which are automatically analyzed using machine learning and computer vision techniques to classify the issue and determine its priority. The system then routes the complaint to the appropriate department, where officers assign tasks to field workers with scheduled visits. Workers resolve the issue, upload proof of completion, and the system updates the complaint status in real time. Citizens are notified throughout the lifecycle and can provide feedback after resolution. The system also includes features such as hotspot detection, analytics dashboards, resource tracking, and duplicate complaint merging, ensuring data-driven decision-making and proactive governance.

The technological stack of the system includes a frontend built with HTML5, CSS3, Bootstrap 5, JavaScript, and libraries like Leaflet.js and Chart.js for interactive maps and visualizations. The backend is implemented using Django (Python), ensuring secure and scalable application logic, while the database uses SQLite for development and PostgreSQL for production environments. The system integrates Scikit-learn for machine learning-based text classification, OpenCV for image analysis, and Gmail SMTP for email notifications. Overall, the system enhances automation, improves efficiency, ensures accuracy in classification and prioritization,

and provides a user-friendly interface for all stakeholders.

**Users of the System**

- ADMIN
- DEPARTMENT OFFICER
- CITIZEN
- FIELD WORKER

**Main Functions**

**Admin Features:**

- **User Management** – Manage all system users including citizens, department officers, and field workers with role-based access control. Admins can create new user accounts, assign roles, and ensure proper authentication and authorization across the system.

- **Complaint Management** – View, monitor, and control all complaints submitted in the system. Admins can update complaint status, assign or reassign complaints to officers and workers, and maintain a complete audit trail of all actions performed.

- **Bulk Operations & Duplicate Handling** – Perform bulk assignment of multiple complaints to workers and merge duplicate complaints into a single primary record to avoid redundancy and improve efficiency.

- **Analytics & Reporting** – Access comprehensive dashboards that display complaint statistics, category distribution, priority levels, monthly trends, and department performance leaderboards. These insights help in data-driven decision-making.

• **Hotspot Detection & Monitoring** – Identify recurring issue locations using geolocation-based analysis. Admins can monitor hotspots and take proactive measures to resolve frequently reported issues.

• **Announcements & Communication** – Post city-wide announcements and updates that are
visible to all users. Notifications are also sent via email to ensure maximum reach.

• **Resource & Cost Management** – Monitor materials used in resolving complaints and track associated costs, enabling financial transparency and efficient budget management.

• **Reschedule Request Handling** – Review and approve or decline reschedule requests submitted by field workers, ensuring proper task scheduling and workflow continuity.

**Department Officer Features:**

- **Complaint Handling** – View and manage complaints related to their assigned department. Officers are responsible for ensuring that complaints are processed efficiently within their domain.

- **Task Assignment** – Assign complaints to field workers based on availability, priority, and location. Officers also schedule visit dates and time slots for task execution.

- **Status Management** – Update the status of complaints as they progress through different stages such as under review, in progress, and completed.

- **Bulk Assignment & Merging** – Perform bulk assignment of complaints and merge duplicate complaints to streamline operations and reduce workload.

- **Reschedule Review** – Evaluate reschedule requests submitted by workers and take appropriate action by approving or rejecting them.

- **Internal Communication** – Communicate with admins and workers through the internal

messaging system for coordination and decision-making.

**Citizen Features:**

- **Registration & Authentication** – Register securely using email-based OTP verification and log in to access personalized services.
- **Complaint Submission** – Submit complaints by providing details such as title, description, category, location, and images. The system automatically classifies the complaint and assigns priority using AI.
- **Complaint Tracking** – Track the real-time status of submitted complaints using a unique complaint ID and view a complete timeline of actions taken.
- **Notifications & Updates** – Receive notifications for status updates, assignment changes, and resolution confirmations via in-app alerts and email.
- **Feedback & Rating** – Provide ratings and feedback after the complaint is resolved, contributing to performance evaluation of departments.
- **Reopen Complaints** – Reopen resolved complaints if the issue persists or the resolution is unsatisfactory, ensuring accountability.
- **Profile Management** – Update personal details such as contact information and address within the system.

**Field Worker Features:**

- **Task Management Dashboard** – View assigned tasks categorized as today's tasks, upcoming tasks, and completed tasks, enabling efficient planning and execution.
- **Complaint Resolution** – Visit the assigned location, resolve the issue, and upload before- and
-after images as proof of completion.

- **Resource Usage Logging** – Record materials used during the resolution process along with quantity and cost, contributing to transparency and auditing.

- **Reschedule Requests** – Request rescheduling of tasks in case of unavailability, providing valid reasons for approval by officers.

- **Status Update through Actions** – Automatically update complaint status to resolved upon uploading resolution proof, ensuring real-time system updates.

- **Internal Messaging** – Communicate with department officers for clarification, coordination, and reporting purposes.

# 2. BACKGROUND

## 2.1 Existing System

In traditional municipal systems, the process of reporting and resolving civic issues is largely manual and unstructured. Citizens typically report problems such as potholes, garbage accumulation, water leakage, or streetlight failures through informal channels including phone calls, direct visits to local offices, or posts on social media platforms. These complaints are often recorded manually or communicated verbally, leading to inconsistencies and lack of proper documentation. There is no centralized platform to register, monitor, or track complaints, which results in fragmented communication between citizens and authorities. Furthermore, the absence of a standardized workflow makes it difficult to assign responsibilities or ensure timely resolution of issues.

The existing system suffers from several inefficiencies due to its dependence on manual processes. Complaints are prone to duplication since there is no mechanism to detect repeated reports from the same location. There is also a lack of prioritization, meaning critical issues such as water pipe bursts near hospitals may not receive immediate attention. Citizens are often unaware of the status of their complaints, leading to dissatisfaction and mistrust in public services. Additionally, there is no data-driven approach to analyze recurring problems or optimize resource allocation. The absence of automation, real-time communication, and accountability mechanisms makes the traditional system inefficient, slow, and unreliable.

## 2.2 Definition of Problem

1. The existing system lacks a centralized digital platform, making it difficult to record, track, and manage complaints efficiently across departments.
2. There is no proper tracking mechanism, which prevents citizens from knowing the status or progress of their submitted complaints.
3. Duplicate complaints are frequently generated due to the absence of location-based validation, leading to redundant work and resource wastage.
4. The system does not support priority-based handling, causing critical issues to be treated the same as minor ones.

5. Manual assignment of tasks leads to delays and inefficiencies in allocating work to field staff.
6. There is no proper accountability, as actions taken on complaints are not recorded systematically.
7. Lack of real-time communication results in poor coordination between citizens, officers, and workers.
8. No analytical tools are available to identify recurring issues or evaluate departmental performance.
9. Resource usage and cost tracking are not maintained, resulting in poor financial transparency.
10. The system lacks security and structured access control, increasing the risk of unauthorized access and data mismanagement.

## 2.3 Proposed System

The proposed **Smart City Issue Reporting & Management System** is a modern, web-based solution designed to overcome the limitations of the traditional system by introducing automation, intelligence, and structured workflows. The system provides a centralized platform where citizens can register and submit complaints with detailed descriptions, geolocation, and image evidence. These complaints are automatically analyzed using machine learning and computer vision techniques to classify the issue and determine its priority level. Based on this analysis, the system intelligently assigns the complaint to the appropriate department, ensuring faster and more accurate routing.

The system establishes a clear role-based workflow involving citizens, administrators, department officers, and field workers. Officers review complaints and assign them to workers with scheduled time slots, while workers resolve issues and upload proof of completion. The system maintains a complete audit trail of all actions, ensuring transparency and accountability. Citizens can track the status of their complaints in real time, receive notifications, and provide feedback after resolution. Additionally, advanced features such as hotspot detection, duplicate complaint merging, analytics dashboards, and resource tracking enable proactive decision-making and efficient resource utilization. By integrating modern technologies and automation, the proposed system significantly improves efficiency, accuracy, and user satisfaction.

## Advantages of Proposed System:

• **Centralized Complaint Management:** Provides a unified digital platform for registering, tracking, and managing complaints, eliminating fragmentation and improving coordination across departments.

• **AI-Based Classification and Prioritization:** Automatically categorizes complaints and assigns priority levels using machine learning, ensuring that critical issues are addressed promptly.

• **Real-Time Tracking and Notifications:** Enables citizens to monitor complaint status and receive instant updates, improving transparency and trust in the system.

• **Efficient Task Assignment:** Automates the assignment of complaints to appropriate departments and workers, reducing manual effort and delays.

• **Duplicate Detection and Merging:** Identifies and merges similar complaints from the same location, minimizing redundancy and optimizing resource usage.

• **Hotspot Identification:** Detects recurring issues based on location data, allowing authorities to take preventive and proactive measures.

• **Resource and Cost Tracking:** Maintains detailed records of materials used and associated costs, ensuring financial accountability and better budget management.

• **Role-Based Access Control:** Ensures secure access by providing different permissions for citizens, admins, officers, and workers, protecting sensitive data.

• **Data-Driven Decision Making:** Provides analytics and performance dashboards that help administrators evaluate trends and improve service delivery.

• **Improved Accountability:** Maintains a complete timeline of actions taken on each complaint, ensuring responsibility and transparency at every stage.

# 3. PROJECT OVERVIEW

## 3.1. Objective of the project

The primary objective of the **Smart City Issue Reporting & Management System** is to develop an intelligent, centralized, and automated platform that enables efficient reporting, tracking, and resolution of civic infrastructure issues within a municipality. The system is designed to replace traditional manual complaint handling methods with a structured, technology-driven workflow that enhances transparency, accountability, and service efficiency. It allows citizens to submit complaints with detailed descriptions, geolocation, and image evidence, which are then automatically analyzed using machine learning and computer vision techniques to classify the issue and determine its priority. The system ensures that complaints are routed to the appropriate departments, assigned to field workers with scheduled timelines, and resolved systematically. Throughout the process, real-time notifications and status tracking keep citizens informed, while feedback mechanisms ensure quality control. By integrating automation, analytics, and role-based operations, the system aims to reduce delays, eliminate redundancy, optimize resource utilization, and improve overall governance and citizen satisfaction.

## 3.2. Stakeholders

**•Citizen**

Citizens are the primary stakeholders who initiate the workflow by reporting civic issues encountered in their surroundings. They interact with the system through registration, complaint submission, and tracking features. Their role involves providing accurate details such as location, description, and supporting images, which helps in proper classification and resolution. Citizens are responsible for monitoring the progress of their complaints, responding to updates, and providing feedback after resolution. Their interaction ensures transparency and accountability, as their feedback contributes to evaluating the performance of departments and field workers.

**•Admin**

The admin acts as the central authority responsible for managing and supervising the entire system. They oversee all complaints, user accounts, and operational workflows. Admins are responsible for assigning or reassigning complaints, managing users across different roles, posting announcements,

and ensuring system-wide coordination. They also analyze data through dashboards, identify recurring issues, monitor departmental performance, and make strategic decisions. Their interaction with the system ensures smooth functioning, policy enforcement, and data-driven improvements.

**•DepartmentOfficer**

Department officers are responsible for managing complaints related to their specific department, such as public works, sanitation, or water supply. They act as intermediaries between the admin and field workers. Their role involves reviewing complaints, updating their status, assigning them to appropriate workers, and ensuring timely resolution. Officers also handle operational tasks such as merging duplicate complaints, performing bulk assignments, and reviewing reschedule requests.

Their interaction ensures efficient task distribution and coordination within departments.

**•FieldWorker**

Field workers are responsible for executing the physical resolution of complaints on the ground. They receive assigned tasks through their dashboard, which includes details such as location, time, and priority. Their responsibilities include visiting the site, resolving the issue, uploading proof of completion in the form of before-and-after images, and logging resource usage. They may also request rescheduling if necessary. Their interaction with the system ensures that complaints are resolved effectively and that all actions are properly documented.

## 3.3. Scope of project

The scope of the Smart City Issue Reporting & Management System encompasses the complete lifecycle of civic complaint management, from submission to resolution and feedback. The system covers functionalities such as user registration with role-based access, complaint submission with geolocation and image upload, AI-based classification and priority prediction, department mapping, task assignment, scheduling, and resolution tracking. It includes advanced features such as duplicate complaint detection, recurring hotspot identification, resource usage tracking, and analytics dashboards for performance evaluation. The system also integrates communication mechanisms like notifications and internal messaging to ensure coordination among stakeholders. While the system primarily focuses on municipal issue management within a city, it is designed to be scalable and adaptable for larger deployments. By automating key processes and providing a centralized platform, the system significantly improves efficiency, reduces manual effort, and enhances transparency compared to traditional methods.

## 3.4. Feasibility Analysis

### 3.4.1 Feasibility study

The development of the Smart City Issue Reporting & Management System is feasible with the available technological resources, tools, and development frameworks. The system leverages widely used and well-supported technologies such as Django, Python, and modern frontend frameworks, which are readily accessible and suitable for building scalable web applications. The required hardware and software resources are minimal and easily available, making the system practical to implement in both development and production environments. Additionally, the modular architecture and clear workflow design ensure that the system can be developed, tested, and deployed efficiently without significant constraints.

### 3.4.2 Technical Feasibility

- **Technology Availability:** The system uses established technologies such as Django (Python), Bootstrap, JavaScript, and PostgreSQL, which are widely

supported and
easily accessible.

- **Data Handling Capability:** The system efficiently manages structured and unstructured data, including complaint details, images, and geolocation data, using optimized database design and indexing.
- **Performance Considerations:** The system incorporates caching, optimized queries,
and scalable architecture to ensure fast response times and efficient handling of multiple users.

The system is technically feasible as it utilizes a robust technology stack consisting of HTML, CSS, Bootstrap, and JavaScript for the frontend, Django (Python) for backend logic, and SQLite/PostgreSQL for database management. The integration of machine learning models and OpenCV enhances functionality without compromising performance. The system is reliable due to Django's built-in security features, scalable through modular design and database optimization, and maintainable due to clean architecture and separation of concerns.

### 3.4.3 Operational Feasibility

The system is operationally feasible as it is designed with a user-friendly interface that accommodates users with varying levels of technical expertise. Citizens can easily register, submit complaints, and track progress without requiring specialized training. Similarly, admins, officers, and workers interact with intuitive dashboards that simplify task management and coordination. The system integrates seamlessly into real-world municipal operations by digitizing existing processes and enhancing them with automation. Minimal training is required for users, making adoption practical and efficient.

### 3.4.4 Schedule Feasibility

The project can be completed within a reasonable timeframe by following a structured development approach that includes requirement analysis, system design, implementation, and testing phases. The modular nature of the system allows parallel

development of components, ensuring timely completion without delays.

Schedule feasibility study for the design is shown below:

| | |
|---|---|
| Problem identification | 3 |
| Requirement analysis | 7 |
| Overall design | 10 |
| Construction | 22 |
| Testing | 10 |

### 3.4.5 Economic Feasibility

The proposed system is economically feasible as it is developed using open-source technologies, which significantly reduces development and deployment costs. The system minimizes manual effort, reduces operational inefficiencies, and optimizes resource utilization, leading to cost savings in the long run. By automating complaint handling, reducing duplication, and improving service efficiency, the system provides a high return on investment. Additionally, improved transparency and accountability contribute to better governance, making the system both cost-effective and beneficial for long-term implementation.

# 4. OVERALL PROJECT PLANNING

## 4.1. Development Environment

**Hardware Specifications**

- Processor: Intel i5 / AMD equivalent or higher
- RAM: Minimum 8GB
- Hard Disk: Minimum 250 GB free space
- Monitor**:** Standard display with 1366x768 resolution or higher
- Input Devices: Keyboard, Mouse
- Other Requirements: Stable internet connection

**Software Specifications**

**Technology used:**

- Front end : HTML5, CSS3, Bootstrap 5, JavaScript, jQuery, Leaflet.js, Chart.js
- IDE : VS Code
- Back end : Django Framework (Python)
- Operating System : Windows
- Tool / IDE :Visual Studio Code

## 4.2. Constraints

The development and implementation of the system are subject to certain limitations and constraints that may affect its performance and usability.

• **Internet Dependency** – The system requires a stable internet connection for accessing features like GPS location, notifications, and data synchronization.

• **AI Model Limitations** – The accuracy of classification and priority prediction depends on trained datasets and may not be perfect in all scenarios.

• **Language Limitation** – The system is primarily optimized for English input, which may affect usability for regional language users.

• **Hardware Dependency** – Efficient performance of the system depends on the availability of adequate hardware resources.

• **Email Service Dependency** – OTP verification and notifications rely on SMTP services, which may fail due to server issues.

## 4.3. Deliverables

The project produces several deliverables that include both system outputs and supporting documentation.

- Fully functional Smart City Issue Reporting & Management System
- User modules: Citizen, Admin, Department Officer, Field Worker
- Complaint management system with AI-based classification and prioritization
- Assignment and scheduling module for task management
- Notification and tracking system
- Analytics dashboard with reports and charts
- Hotspot detection and duplicate complaint handling features

• Resource usage and cost tracking module
• Internal messaging system
• Database schema and system design documentation
• Project report including all phases and diagrams

## 4.4. Assumptions and Dependencies

### a) Assumptions

• Users have basic knowledge of using web applications such as login, form submission, and navigation.

• Citizens will provide accurate and complete information while submitting complaints.

• The system will be accessed through devices with internet connectivity.

• All users will follow their assigned roles and responsibilities within the system.

• AI models will provide reasonably accurate predictions based on available data.

### b) Dependencies

• Reliable internet connection for accessing system features and real-time updates.

• Availability of web browsers compatible with modern web technologies.

• Backend server environment capable of running Django applications.

• External services such as Gmail SMTP for sending emails and notifications.

• Libraries and frameworks like Scikit-learn and OpenCV for AI functionality.

## 4.5. Risks

The project may encounter several risks that could impact its development and performance.

- **System Downtime** – Server or network failures may temporarily disrupt system availability.
- **Incorrect AI Predictions** – AI models may misclassify complaints, affecting priority and routing.
- **Security Risks** – Unauthorized access or data breaches may occur if proper security measures are not enforced.
- **Data Loss** – Improper handling or system crashes may lead to loss of complaint or user data.
- **User Adoption Issues** – Users unfamiliar with digital systems may face difficulty in initial usage.

## 4.6. Process Model

The development of the system follows a structured Waterfall Model, where each phase is completed before moving to the next. This model ensures systematic planning, clear documentation, and proper validation at each stage of development.

- Requirement analysis
- System study
- Designing
- Coding
- Testing
- Maintenance

## 4.7. Test Strategy

### 4.7.1 System Testing

System testing is performed to ensure that the entire application functions as expected in an integrated environment. It verifies that all modules work together correctly and that the system meets the

specified requirements. The testing process focuses on validating the workflow of complaint submission, assignment, resolution, and feedback.

- **Validation of system functionality** – Ensures all features such as complaint submission, tracking, and resolution operate correctly.
- **Role-based testing** – Verifies that each user role (Citizen, Admin, Officer, Worker) can access only their permitted functionalities.
- **Input-output verification** – Confirms that valid inputs produce correct outputs and invalid inputs are handled properly.
- **Security and process validation** – Ensures secure login, data protection, and proper workflow execution.

System testing ensures that the application is stable, reliable, and ready for deployment by covering all major functionalities and interactions between modules.

### 4.7.2 Types of Testing

**Unit Testing**

Unit testing involves testing individual components or functions of the system independently. Each function, such as complaint submission or user authentication, is tested to ensure it performs as expected. This helps identify errors at an early stage.

**Module Testing**

Module testing focuses on testing individual modules such as accounts, complaints, AI engine, and messaging. Each module is tested for its internal functionality and correctness before integration with other modules.

**Integration Testing**

Integration testing ensures that different modules of the system work together seamlessly. For example, the interaction between complaint submission, AI classification, and assignment modules is tested to verify proper data flow.

**Validation Testing**

Validation testing checks whether the system meets user requirements and expectations. It ensures that all functionalities align with the project objectives and provide the intended outcomes.

**Output Testing**

Output testing verifies that the results generated by the system, such as complaint reports, notifications, and analytics, are accurate and correctly formatted.

**Acceptance Testing**

Acceptance testing is performed to confirm that the system is ready for deployment and meets user expectations. It involves testing the system in real-world scenarios to ensure usability, performance, and reliability.

## 4.8 Testing environment and tools

The hardware specification used for testing:

| | |
|---|---|
| Operating system | Windows 11 |
| Memory | 8 GB |
| Hard Disk | 250 GB minimum |

The software specification used for testing:

| | |
|---|---|
| Front End | HTML5, CSS3, Bootstrap 5, JavaScript, jQuery |
| Back End | Django (Python) |
| Operating System | Windows 11 |

# 5. ITERATION PLANNING

## 5.1 Schedule

| SERIAL NO. | TASK | DURATION |
|---|---|---|
| 1 | Problem identification | 3 days |
| 2 | Requirement Specification | 7 days |
| 3 | Database Design and Analysis | 8 days |
| 4 | Design Analysis | 6 days |
| 5 | Coding | 20 days |
| 6 | Testing | 8 days |
| | Total | 52 days |

## 5.2 Risk

• **Incorrect Data Input:** Users may enter incomplete or incorrect complaint details, which can affect AI classification and delay proper resolution.

• **AI Prediction Errors:** Machine learning models may misclassify complaints or assign incorrect priorities, impacting task routing and urgency handling.

• **System Downtime:** Server or network failures may interrupt complaint submission, tracking, or notification services.

• **Database Failure:** Data corruption or loss may occur due to unexpected crashes, affecting complaint records and user information.

• **Security Vulnerabilities:** Unauthorized access or weak authentication mechanisms may lead to data breaches or misuse of the system.

• **Dependency on External Services:** Email notifications rely on SMTP services, and failure of these services may affect communication with users.

• **Scalability Issues:** Increased number of users or complaints may affect system performance if not properly optimized.

• **User Adoption Challenges:** Users unfamiliar with digital platforms may face difficulties in using the system effectively.

# 6. HIGH LEVEL SYSTEM ANALYSIS

## 6.1 User Characteristics

The users of the system are expected to have basic knowledge of computers and internet usage. They should be familiar with common web-based operations such as logging in, filling forms, uploading files, and navigating dashboards. The system is designed to be user-friendly so that even individuals with minimal technical expertise can interact with it efficiently.

- Citizen
- Admin
- Department Officer
- Field Worker

## 6.2 Summary of System Features/Functional Requirements

**User Management**

This feature allows the system to manage different types of users with role-based access control. It includes registration with OTP verification, secure login, profile management, and role-based dashboards. Each user can access only the functionalities permitted to their role.

**Complaint Management**

The system enables citizens to submit complaints with detailed information such as title, description, images, and location. Complaints are stored, tracked, and managed throughout their lifecycle, from submission to resolution.

**AI-Based Classification and Prioritization**

The system uses machine learning and computer vision techniques to automatically classify complaints into categories and assign priority levels. This reduces manual effort and ensures critical issues are handled promptly.

**Task Assignment and Scheduling**

Complaints are assigned to field workers by admins or officers with specific dates and time slots. The system ensures no worker is double-booked and helps in efficient task management.

**Notification and Tracking System**

Users receive real-time notifications regarding complaint status updates, assignments, and resolutions. Citizens can track their complaints using a unique complaint ID and view a detailed timeline.

**Duplicate Detection and Merging**

The system identifies complaints reported from nearby locations and allows merging of duplicates to avoid redundancy and optimize resource utilization.

**Hotspot Detection**

Recurring complaints from the same location are identified as hotspots. This feature helps administrators take proactive measures to address frequently occurring issues.

**Resource and Cost Management**

Field workers can log materials used during issue resolution along with their costs. This ensures transparency and helps in tracking resource utilization.

**Analytics and Reporting**

The system provides dashboards with charts and reports showing complaint statistics, trends, and department performance. This supports data-driven decision-making.

**Internal Messaging System**

A messaging module enables communication between admins, officers, and workers, improving coordination and reducing delays.

## 6.3 Non Functional Requirements/Supplementary Specifications

The system must meet several non-functional requirements to ensure performance, security, and usability.

**Accuracy:**

The system ensures accurate classification of complaints and correct assignment of priority levels using AI models. Data validation mechanisms ensure that user inputs are correctly processed.

**Reliability:**

The system is designed to operate consistently without failure. It maintains data integrity and ensures that all complaint records and updates are stored securely.

**Responsiveness:**

The application provides quick responses to user actions such as form submission, tracking, and dashboard loading, ensuring a smooth user experience.

**Usability:**

The system offers a clean and intuitive interface with simple navigation, making it easy for users to interact with the platform without extensive training.

**Other non-functional requirements are:**

• **Security** – Implements authentication, authorization, and protection against common web vulnerabilities to safeguard user data.

• **Maintainability** – The modular design allows easy updates, debugging, and enhancement of system components.

• **Extensibility** – The system can be expanded with additional features such as mobile integration or advanced AI models.

• **Reusability** – Code components and modules can be reused for similar applications or future developments.

• **Resource Utilization** – Efficient use of system resources ensures optimal performance without excessive load.

## 6.4 Glossary

| | |
|---|---|
| Complaint | A reported civic issue submitted by a citizen |
| AI Classification | Process of categorizing complaints using machine learning |
| Priority | Level of urgency assigned to a complaint |
| Hotspot | A location with recurring complaints |

| | |
|---|---|
| Assignment | Allocation of a complaint to a field worker |

## 6.5 Business Rules

- A citizen can submit multiple complaints but can only view their own complaints.
- Each complaint must have a unique complaint ID generated automatically.
- Complaints must be assigned to a department based on category.
- A worker cannot be assigned multiple tasks at the same time slot.
- A complaint can be marked as resolved only after uploading resolution proof.
- Feedback can be given only after a complaint is resolved.
- Duplicate complaints can be merged into a single primary complaint.
- Only authorized roles can update complaint status or assign tasks.

## 6.6 Use Case

**Login**

Users enter their valid username and password to access the system. Based on their role (Citizen, Admin, Officer, Worker), they are redirected to their respective dashboards.

**Register**

New users create an account by providing required details and verifying their identity using OTP-based email authentication.

**Submit Complaint**

Citizens report civic issues by entering title, description, category, location, and uploading images. The system processes the complaint using AI for classification and priority assignment.

**Track Complaint**

Citizens can track the real-time status of their complaints using a unique complaint ID and view the progress timeline.

**View Complaint Details**

Users can view complete information of a complaint including status, assigned staff, images, timeline updates, and resolution proof.

**Reopen Complaint**

Citizens can reopen a complaint if the issue is not properly resolved by providing a valid reason.

**Download Complaint PDF**

Citizens can download a detailed PDF report of their complaint for record-keeping or official purposes.

**Update Profile**

Users can update their personal information such as name, contact details, and profile picture.

**Assign Complaint**

Admins and officers assign complaints to field workers with specific dates and time slots, ensuring proper scheduling and workload management.

**Bulk Assign Complaints**

Admins and officers can assign multiple complaints at once to a worker, improving efficiency during high workload situations.

**Update Complaint Status**

Admins and officers can update complaint status at various stages such as under review, in progress, or completed.

**Merge Duplicate Complaints**

Admins and officers can merge multiple complaints related to the same issue into a single primary complaint to avoid redundancy.

**Manage Users**

Admins can create, view, and manage users including citizens, officers, and workers with proper role-based permissions.

**Post Announcements**

Admins can publish important updates or notices that are visible to all users in the system.

**View Analytics**

Admins and officers can analyze complaint data through dashboards showing trends, performance, and statistics.

**View Hotspots**

Admins can identify recurring issue locations based on complaint frequency and take proactive actions.

**View Complaint Map**

Admins can visualize all complaints on a map using location data for better geographical analysis.

**Review Reschedule Requests**

Admins and officers review requests submitted by workers for rescheduling assigned tasks and take appropriate action.

**View Assigned Tasks**

Field workers can view their assigned tasks categorized into today's, upcoming, and completed tasks.

**Upload Resolution Proof**

Field workers upload before-and-after images and notes after resolving a complaint, which updates the complaint status.

**Add Resource Usage**

Field workers record materials used and their costs during issue resolution for transparency and tracking.

**Request Reschedule**

Field workers can request rescheduling of assigned tasks by providing valid reasons.

**AI Analyze Complaint**

The system automatically analyzes complaint text and images to classify the issue and assign priority using machine learning and computer vision.

**Detect Nearby Complaints**

The system checks for existing complaints in nearby locations to prevent duplicate submissions.

**Hotspot Detection**

The system identifies locations with recurring complaints and flags them as hotspots for proactive management.

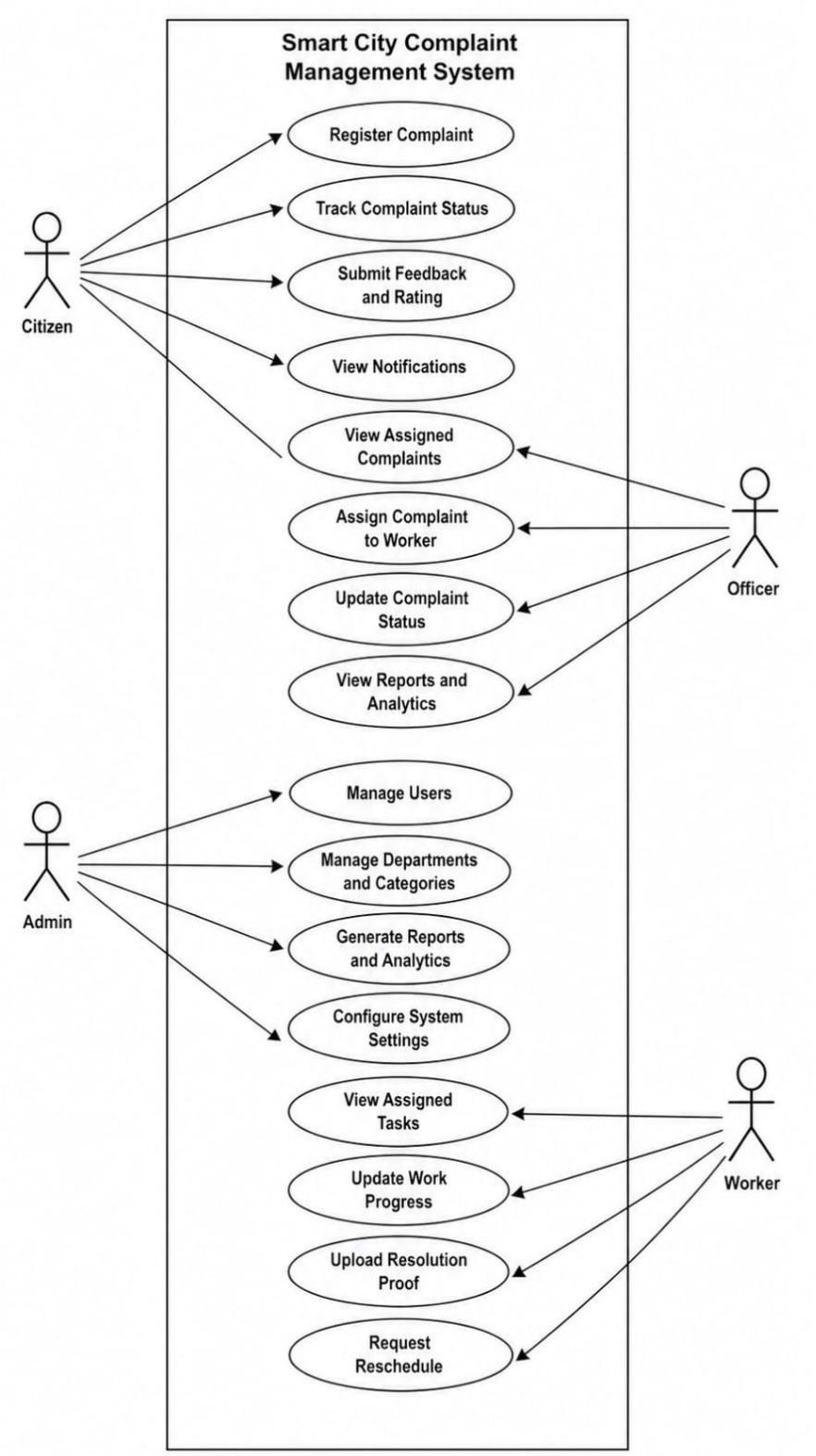
**Send Notifications**

The system sends real-time notifications to users regarding complaint submission, status updates, assignments, and resolution.

**Internal Messaging**

Admins, officers, and workers can communicate with each other through the internal messaging system for coordination and task management.

## 6.7 Use -Case Diagram



# 7. USE CASE MODEL

## 7.1. Use Case Text

**Scope:** Smart City Issue Reporting & Management System

**Primary Actor:** Citizen, Admin, Department Officer, Field Worker

**Stakeholders and Interests:**

- **Citizen:**

Citizens are the primary users who interact with the system to report civic issues. Their responsibility is to provide accurate complaint details including location, description, and images. They track complaint status, receive notifications, and provide feedback after resolution. Their interaction ensures transparency and accountability in the system.

- **Admin:**

The admin oversees the entire system and manages all operations. Their responsibilities include managing users, monitoring complaints, assigning tasks, analyzing data, and posting announcements. Admins interact with all modules to ensure proper functioning, efficient workflow, and decision-making based on analytics.

- **Department Officer:**

Department officers handle complaints related to their specific departments. They are responsible for reviewing complaints, assigning them to field workers, updating statuses, and managing scheduling. They interact with both admins and workers to ensure timely resolution of issues.

- **Field Worker:**

Field workers are responsible for executing assigned tasks on the ground. They receive complaint assignments, visit locations, resolve issues, upload proof, and log resource usage.

Their interaction ensures that complaints are physically addressed and properly documented.

**Preconditions:**

Users must be registered and authenticated in the system, and the system must be operational with proper network connectivity.

**Success Guarantee (Post conditions):**

After successful operation, the system ensures that the complaint is properly recorded, classified, assigned to the appropriate department and worker, resolved with proof of completion, and updated in the system with full traceability. Citizens receive timely notifications and can provide feedback, while all actions are logged to maintain transparency, accountability, and data consistency across the system.

**Main Success Scenario:**

1. The citizen registers and logs into the system.
2. The citizen submits a complaint with details such as title, description, location, and images.
3. The system processes the complaint using AI to classify the category and assign priority.
4. The complaint is automatically mapped to the appropriate department.
5. The admin or department officer reviews the complaint.
6. The officer/admin assigns the complaint to a field worker with a scheduled date and time.
7. The field worker receives the task notification and views it in the dashboard.
8. The field worker visits the location and resolves the issue.
9. The worker uploads before-and-after images as proof and logs resource usage.
10. The system updates the complaint status to resolved and notifies the citizen.
11. The citizen views the resolution details and provides feedback and rating.
12. The admin monitors analytics and performance based on completed complaints.

**Extensions:**

1. If the user enters incorrect login credentials, the system displays an error message and denies access.
2. If OTP verification fails during registration, the user must retry the process.
3. If AI classification fails, the system falls back to rule-based classification.
4. If a worker is unavailable, the officer reschedules or assigns another worker.
5. If duplicate complaints are detected, the system allows merging into a primary complaint.
6. If a citizen is not satisfied with resolution, they can reopen the complaint.

**Special Requirements:**

1. The system must provide a responsive and user-friendly interface accessible across devices.
2. The system must ensure secure authentication, role-based access control, and protection against unauthorized access.
3. The system should support real-time notifications, efficient data handling, and integration with AI models and external services.

**Frequency of Occurrence:**

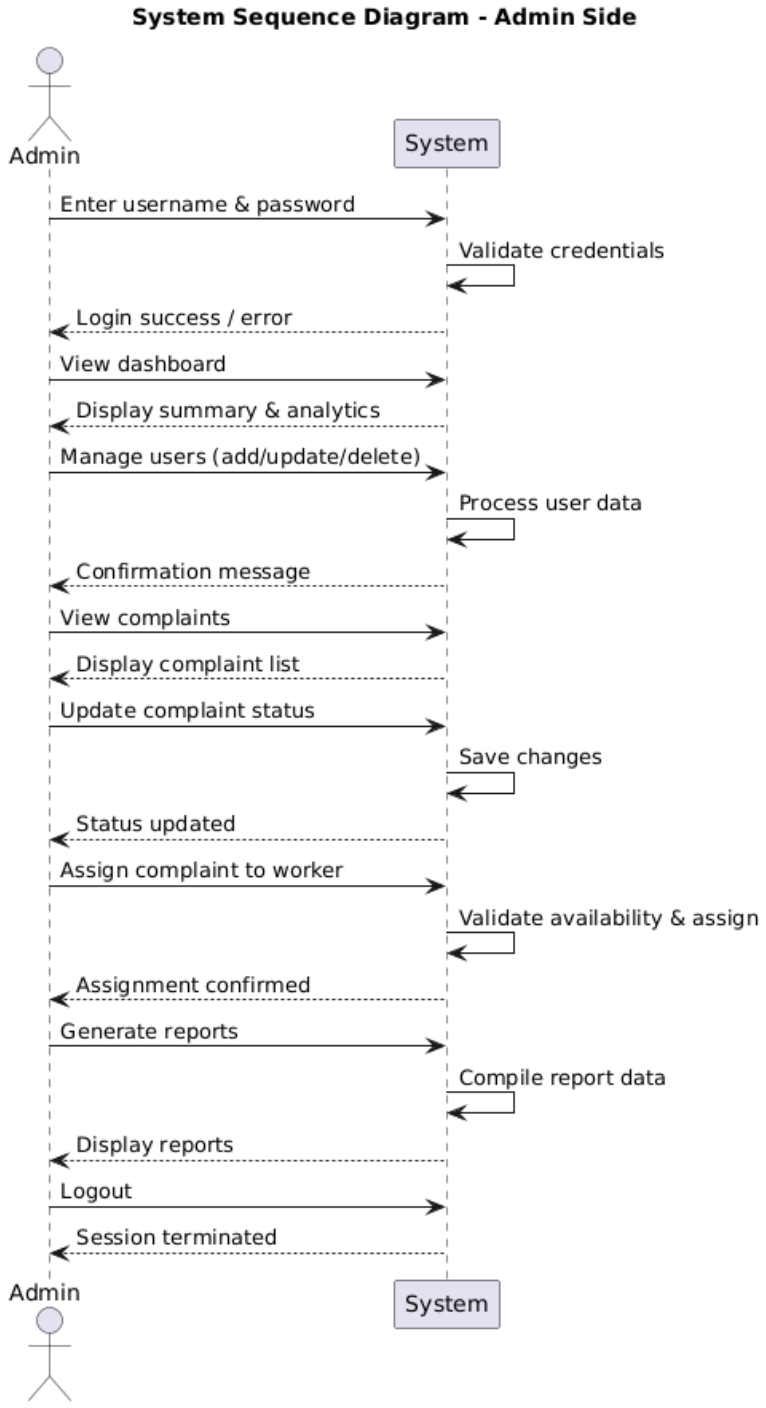
- The system is used continuously by multiple users for complaint submission, tracking, and management.
- Peak usage occurs during emergencies, public issues, or high complaint periods such as after heavy rains or city events.

**Open Issues:**

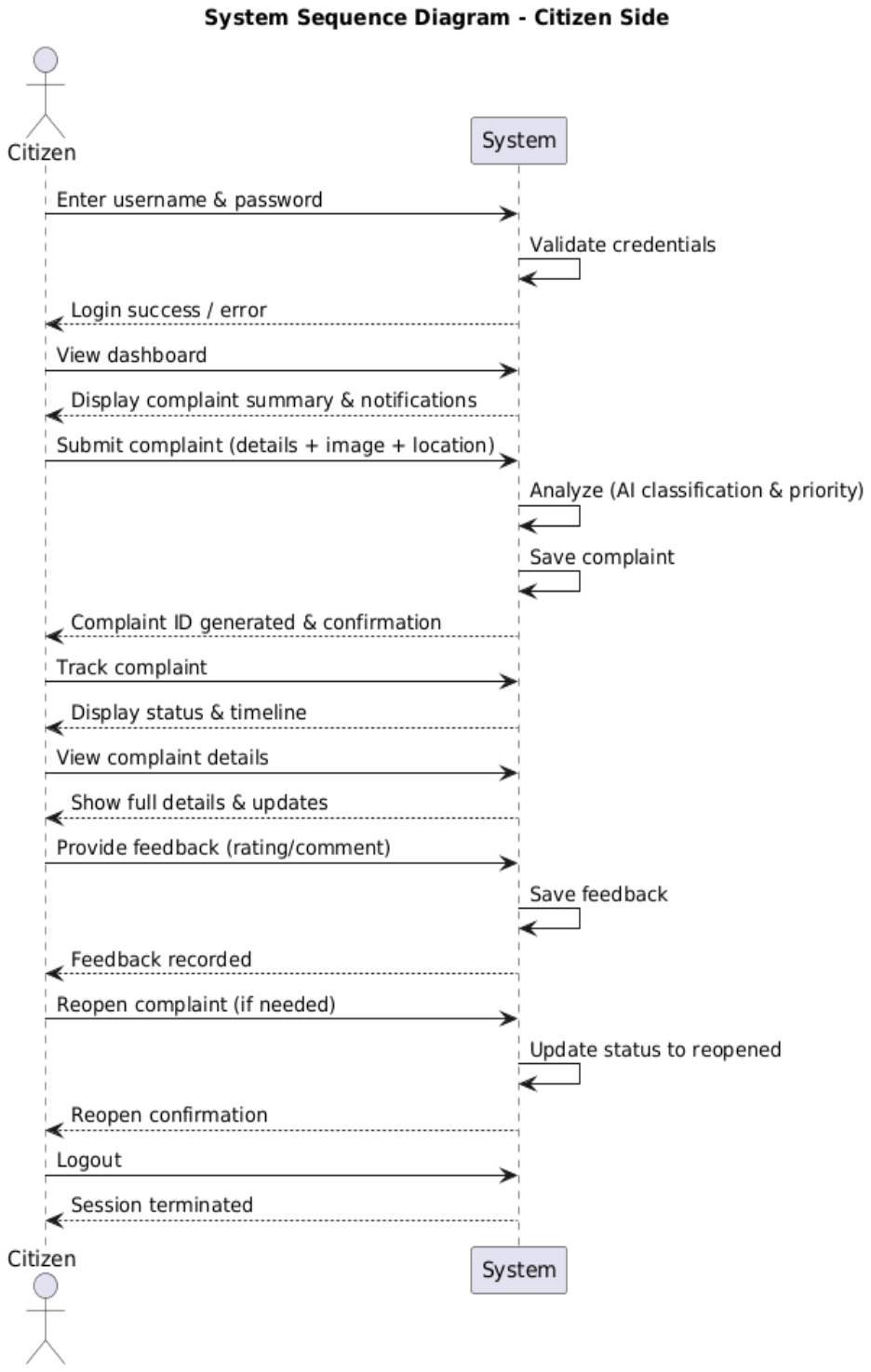
1. Accuracy of AI models may vary depending on training data and real-world scenarios.
2. System performance may be affected under heavy load if scalability measures are not implemented.
3. Future enhancements such as mobile app integration and multilingual support require additional development and resources.

## 7.2 System Sequence Diagram

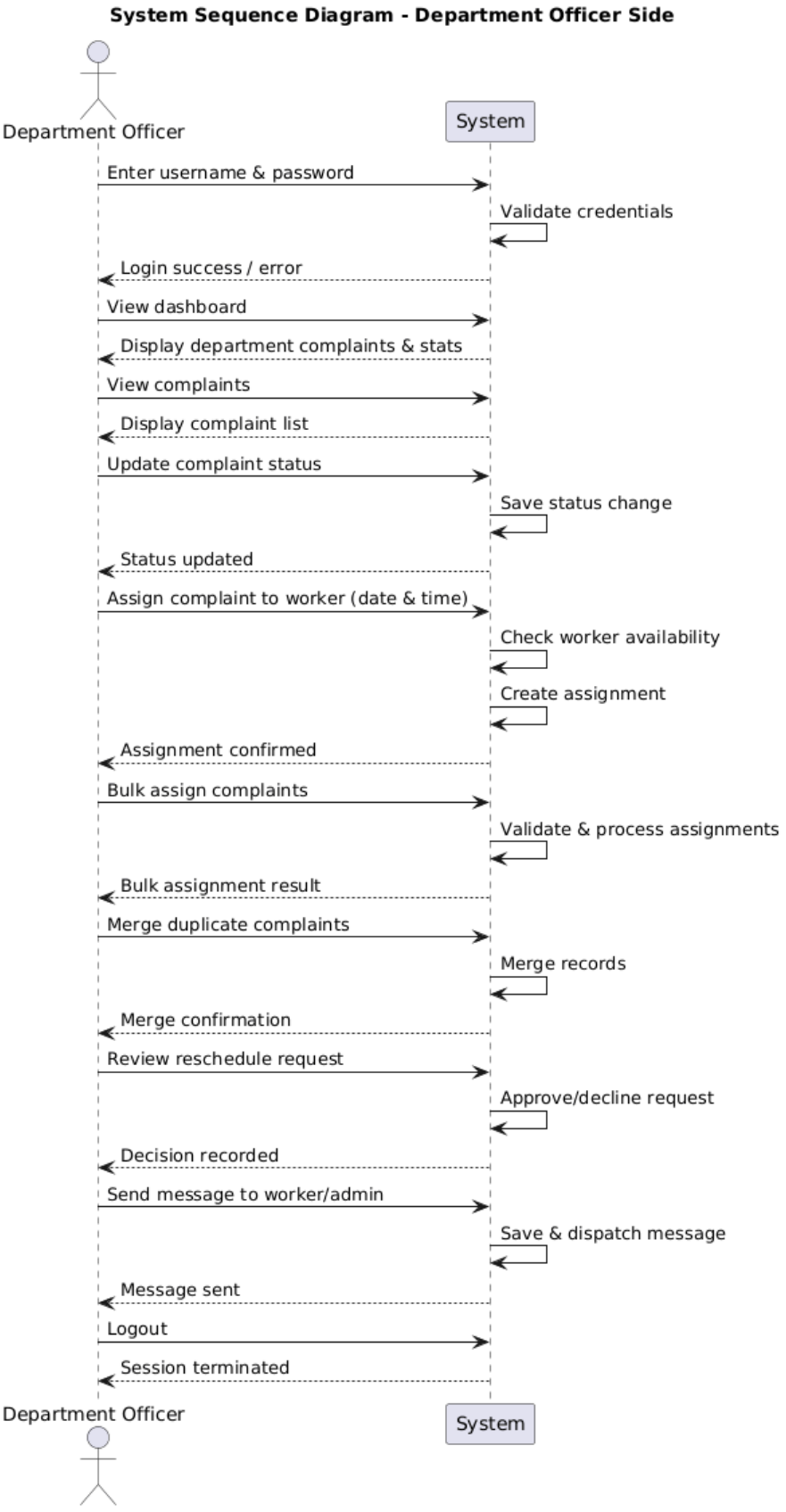
### Admin Side



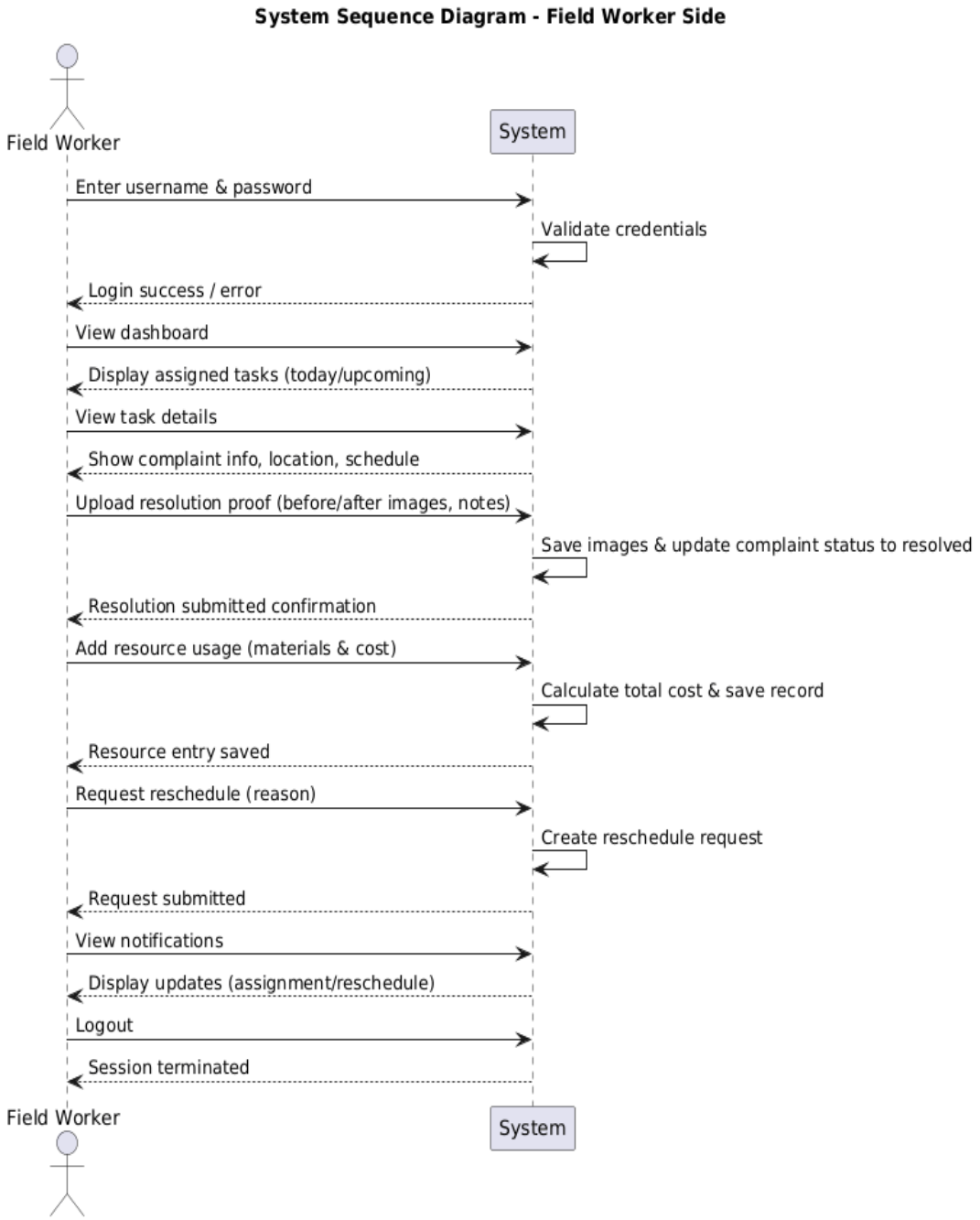
## Citizen Side



## Department Officer Side



## Field Worker Side



## 7.3 Operation Contracts

**Operation:** Citizen (username: string, password: string)

**Cross Reference:** Use case: Citizen login

**Preconditions:**

- Citizen must be registered and have a verified account in the system

**Conditions:**

•Login instance created

• Credentials validated/verified

• Access rights and actions available to citizen such as submitting complaints, tracking status, and providing feedback

**Operation:** Admin (username: string, password: string)

**Cross Reference:** Use case: Admin login

**Preconditions:**

- Admin account must exist with authorized privileges

**Conditions:**

• Login instance created

• Credentials validated/verified

• Access rights and actions available to admin such as managing users, assigning complaints, viewing analytics, and posting announcements

**Operation:** Department Officer (username: string, password: string)

**Cross Reference:** Use case: Officer login

**Preconditions:**

- Officer must be registered and assigned to a department

**Conditions:**

• Login instance created

• Credentials validated/verified

• Access rights and actions available to officer such as assigning tasks, updating complaint status, and managing department-related complaints

**Operation:** Field Worker (username: string, password: string)

**Cross Reference:** Use case: Worker login

**Preconditions:**

- Worker account must be created and assigned tasks by admin or officer

**Conditions:**

• Login instance created

• Credentials validated/verified

• Access rights and actions available to worker such as viewing assigned tasks, uploading resolution proof, and requesting rescheduling

### 7.4 Reports

**Complaint Report:**

This report contains detailed information about all complaints including complaint ID, category, priority, status, assigned staff, and timestamps. It is used to monitor complaint lifecycle and evaluate system performance.

**Status Report:**

This report provides a summary of complaints based on their current status such as submitted, in progress, and resolved. It helps administrators track progress and identify pending issues.

**User Report:**

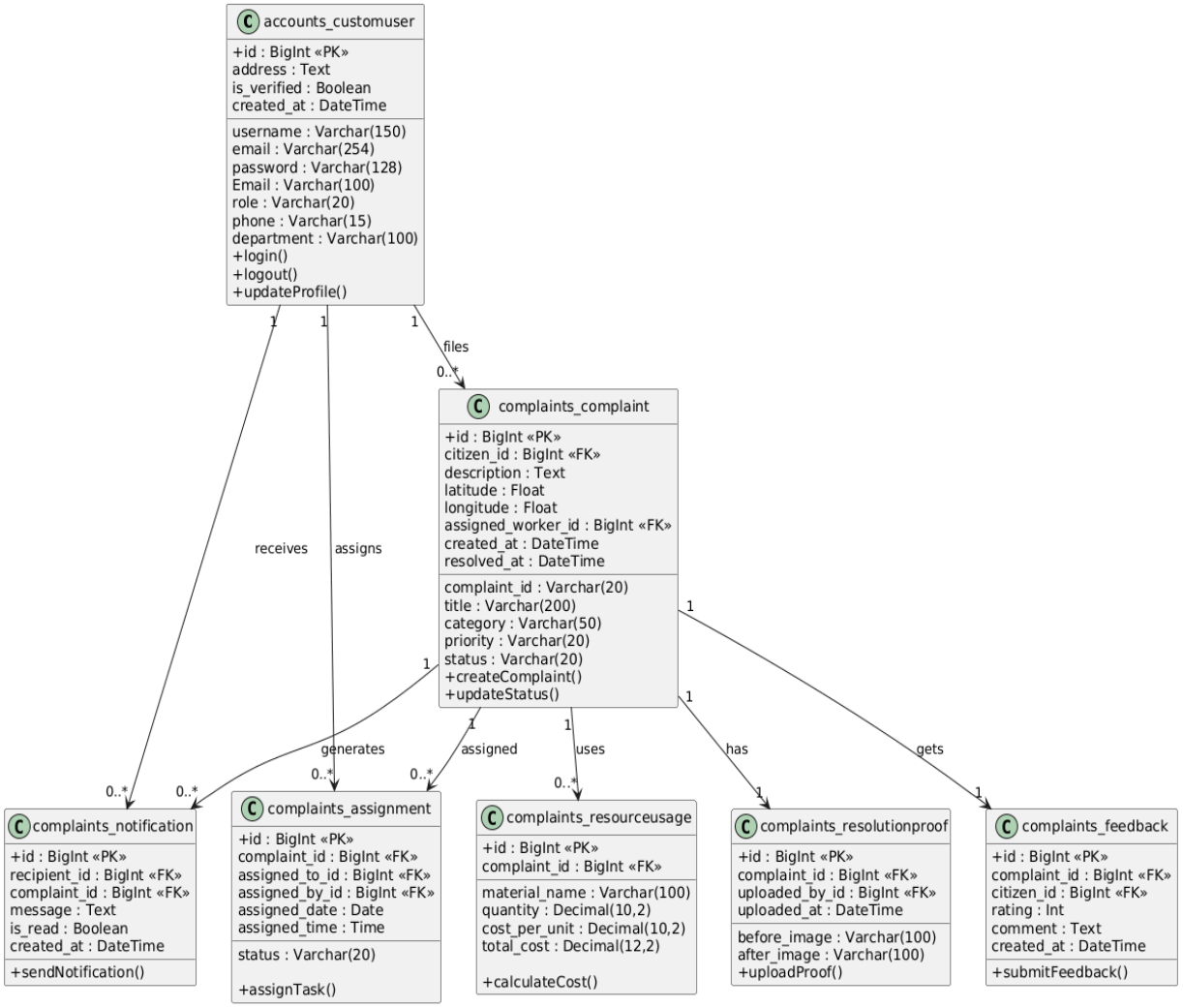
This report includes details of all users such as citizens, officers, and workers along with their roles and activity levels. It is useful for managing user accounts and analyzing system usage.

**Department Performance Report:**

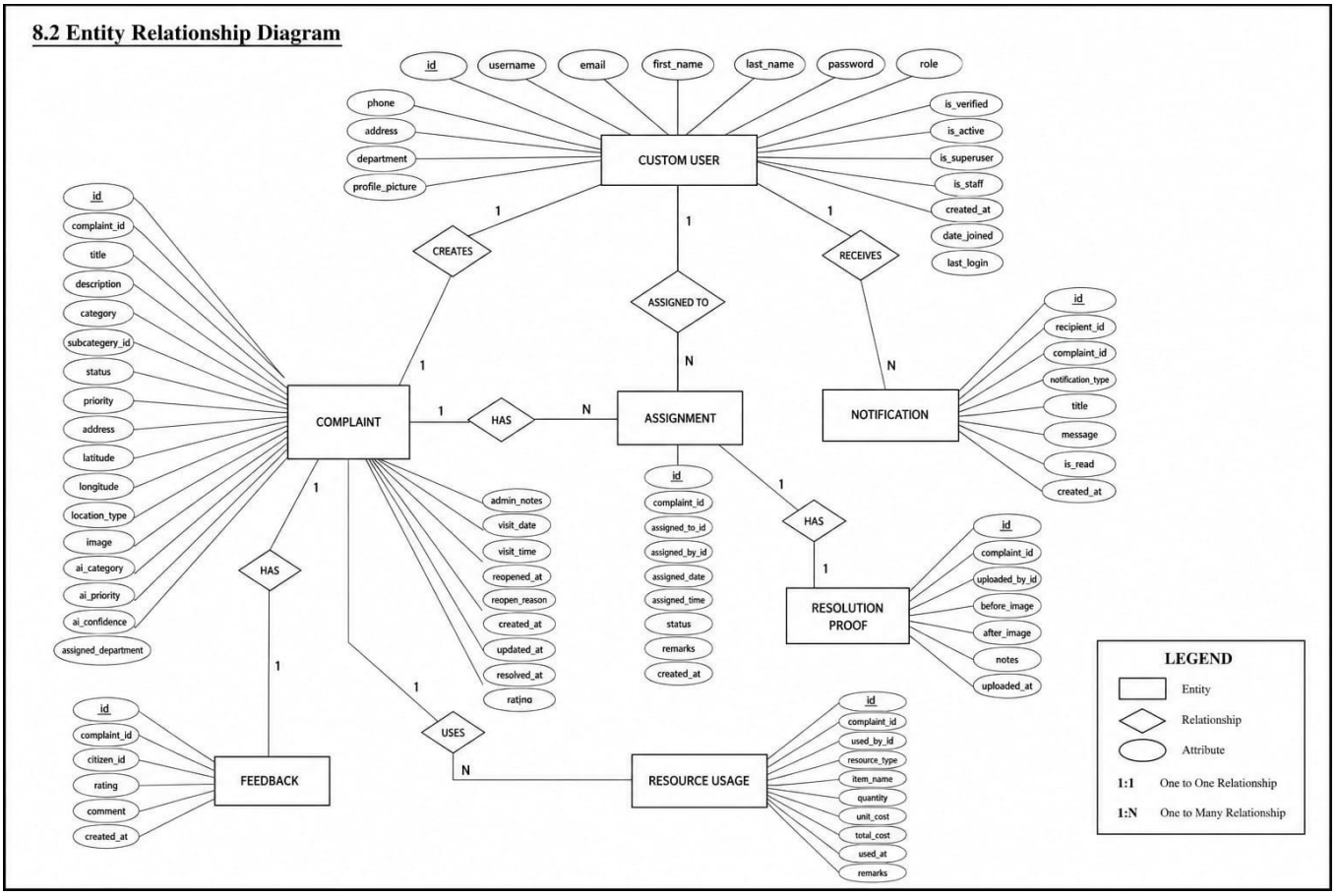
This report evaluates the performance of different departments based on metrics such as average resolution time, number of resolved complaints, and user feedback ratings. It helps in identifying efficient and underperforming departments.

# 8. DESIGN MODEL

## 8.1 Class Diagram



## 8.2 Entity Relationship Diagram



## 8.3 UI Design

**ADMIN:**

**Login Page:** Admin enters username and password to securely access the system dashboard.

**Dashboard:** Displays overall system statistics including total complaints, pending issues, resolved cases, and performance insights for monitoring operations.

**User Management Page:** Allows admin to add, edit, and manage users including citizens, officers, and field workers with role-based access control.

**Complaint Management Page:** Enables viewing, filtering, updating status, assigning complaints, and merging duplicate complaints.

**Reports & Analytics Page:** Provides graphical insights such as complaint trends, department performance, and hotspot analysis.

**Announcement Page:** Allows admin to post system-wide announcements and notifications to users.

**Logout Page:** Securely logs out the admin and ends the session.

**DEPARTMENT OFFICER:**

**Login Page:** Officer enters valid credentials to access the department-specific dashboard.

**Dashboard:** Displays complaints related to the assigned department along with status summaries and priority alerts.

**Complaint Review Page:** Allows officers to view, filter, and update complaint status such as under review or in progress.

**Assignment Page:** Enables assigning complaints to field workers with date and time scheduling.

**Bulk Assignment Page:** Allows assigning multiple complaints simultaneously for efficient workload management.

**Reschedule Management Page:** Provides functionality to review and approve or reject worker reschedule requests.

**Logout Page:** Ends the session securely after operations.

**FIELD WORKER:**

**Login Page:** Field worker logs in using credentials to access assigned tasks.

**Dashboard:** Displays assigned complaints categorized into today's, upcoming, and completed tasks.

**Task Detail Page:** Shows complete complaint information including location, priority, and instructions.

**Status Update Page:** Allows uploading resolution proof (before/after images) and updating task status.

**Resource Entry Page:** Enables logging of materials used and associated costs during resolution.

**Reschedule Request Page:** Allows workers to request changes in assigned schedule with valid reasons.

**Logout Page:** Ends the session securely.

**CITIZEN:**

**Login Page:** Customer registers or logs in using credentials with OTP verification for secure access.

**Dashboard:** Displays summary of submitted complaints, recent updates, and notifications.

**Complaint Submission Page:** Allows users to enter complaint details such as title, description, category, location (map-based), and upload images with AI-assisted suggestions.

**Complaint Tracking Page:** Enables users to track complaint status and view progress timeline.

**Complaint Detail Page:** Displays full complaint information including assigned staff, updates, and resolution proof.

**Feedback Page:** Allows users to provide ratings and comments after complaint resolution.

**Reopen Complaint Page:** Enables users to reopen complaints if not satisfied with the resolution.

**Logout Page:** Securely ends the session.

## 8.4 Theoretical Background

The system is developed using a combination of modern web technologies that ensure efficient communication between users and the application. The frontend is built using HTML, CSS, Bootstrap, and JavaScript to provide a responsive and interactive user interface, enabling users to submit and track data through forms, dashboards, and visual components. The backend is implemented using a server-side framework that handles application logic, request processing, and interaction with the database. A relational database management system is used to store structured data such as user details, records, assignments, and system logs in an organized manner. The system follows a request-response communication model in which client requests are processed by the server, necessary operations are performed, and responses are returned dynamically. Advanced processing logic is incorporated to analyze user inputs and automate certain decisions, improving system efficiency and reducing manual intervention.

The system ensures secure user authentication through credential validation and verification mechanisms, allowing only authorized users to access the application. Role-based access control is implemented so that different users interact with the system according to their permissions and responsibilities. Data handling is managed through structured database operations, ensuring consistency and integrity of stored information. The system also supports file and image handling, allowing users to upload and store media data, which is later processed and retrieved as needed. Various operations such as validation, scheduling, and workflow execution are handled through defined processing logic to maintain accuracy and efficiency. Security is maintained through input validation, session management, and controlled access to system resources. The development environment includes tools such as integrated development environments or code editors, a local server setup for testing and deployment, and a compatible operating system, all of which support efficient development, debugging, and maintenance of the application.

## 8.5 Database Design

Table Name: accounts_customuser

| Field Name | Data Type | Description | Constraints |
|---|---|---|---|
| id | BigInt | Unique user identifier | Primary Key, Auto Increment |
| username | Varchar(150) | Username for login | Unique, Not Null |
| email | Varchar(254) | User email | Not Null |
| password | Varchar(128) | Encrypted password | Not Null |
| role | Varchar(20) | User role (citizen/admin/officer/worker) | Default 'citizen' |
| phone | Varchar(15) | Contact number | Optional |
| address | Text | User address | Optional |
| department | Varchar(100) | Department name | Optional |
| is_verified | Boolean | Email verification status | Default True |
| created_at | DateTime | Account creation time | Auto |

Table name: complaints_complaint

| Field Name | Data Type | Description | Constraints |
|---|---|---|---|
| id | BigInt | Unique record ID | Primary Key |
| complaint_id | Varchar(20) | Complaint reference ID | Unique, Not Null |
| citizen_id | BigInt | Reference to user | Foreign Key |
| title | Varchar(200) | Complaint title | Not Null |
| description | Text | Complaint details | Not Null |
| category | Varchar(50) | Complaint category | Default 'other' |
| priority | Varchar(20) | Complaint priority | Default 'medium' |
| status | Varchar(20) | Current status | Default 'submitted' |
| latitude | Float | Location latitude | Optional |
| longitude | Float | Location longitude | Optional |
| assigned_worker_id | BigInt | Assigned worker | Foreign Key, Nullable |

| Field Name | Data Type | Description | Constraints |
|---|---|---|---|
| created_at | DateTime | Submission time | Auto |
| resolved_at | DateTime | Resolution time | Nullable |

Table Name: complaints_assignment

| Field Name | Data Type | Description | Constraints |
|---|---|---|---|
| id | BigInt | Assignment ID | Primary Key |
| complaint_id | BigInt | Related complaint | Foreign Key |
| assigned_to_id | BigInt | Worker assigned | Foreign Key |
| assigned_by_id | BigInt | Officer/Admin assigning | Foreign Key |
| assigned_date | Date | Scheduled date | Optional |
| assigned_time | Time | Scheduled time | Optional |
| status | Varchar(20) | Assignment status | Default 'pending' |

Table Name: complaints_resolutionproof

| Field Name | Data Type | Description | Constraints |
|---|---|---|---|
| id | BigInt | Unique ID | Primary Key |
| complaint_id | BigInt | Related complaint | One-to-One, Foreign Key |
| uploaded_by_id | BigInt | Worker ID | Foreign Key |
| before_image | Varchar(100) | Image before fix | Optional |
| after_image | Varchar(100) | Image after fix | Optional |
| uploaded_at | DateTime | Upload timestamp | Auto |

Table Name: complaints_feedback

| Field Name | Data Type | Description | Constraints |
|---|---|---|---|
| id | BigInt | Feedback ID | Primary Key |
| complaint_id | BigInt | Related complaint | One-to-One, Foreign Key |
| citizen_id | BigInt | Citizen who gave feedback | Foreign Key |
| rating | Int | Rating (1–5) | Constraint (1–5) |
| comment | Text | Feedback text | Optional |
| created_at | DateTime | Feedback time | Auto |

Table Name: complaints_notification

| Field Name | Data Type | Description | Constraints |
|---|---|---|---|
| id | BigInt | Notification ID | Primary Key |
| recipient_id | BigInt | User receiving notification | Foreign Key |
| complaint_id | BigInt | Related complaint | Foreign Key |
| message | Text | Notification content | Not Null |
| is_read | Boolean | Read/unread status | Default False |
| created_at | DateTime | Timestamp | Auto |

Table Name: complaints_resourceusage

| Field Name | Data Type | Description | Constraints |
|---|---|---|---|
| id | BigInt | Resource ID | Primary Key |
| complaint_id | BigInt | Related complaint | Foreign Key |
| material_name | Varchar(100) | Material used | Not Null |
| quantity | Decimal(10,2) | Quantity used | Not Null |
| cost_per_unit | Decimal(10,2) | Cost per unit | Default 0 |
| total_cost | Decimal(12,2) | Total calculated cost | Auto Calculated |

# 9. TESTING

## 9.1 Test cases

Test Scenario: Checking Login Functionality

**Test Case 1: Invalid User Login**

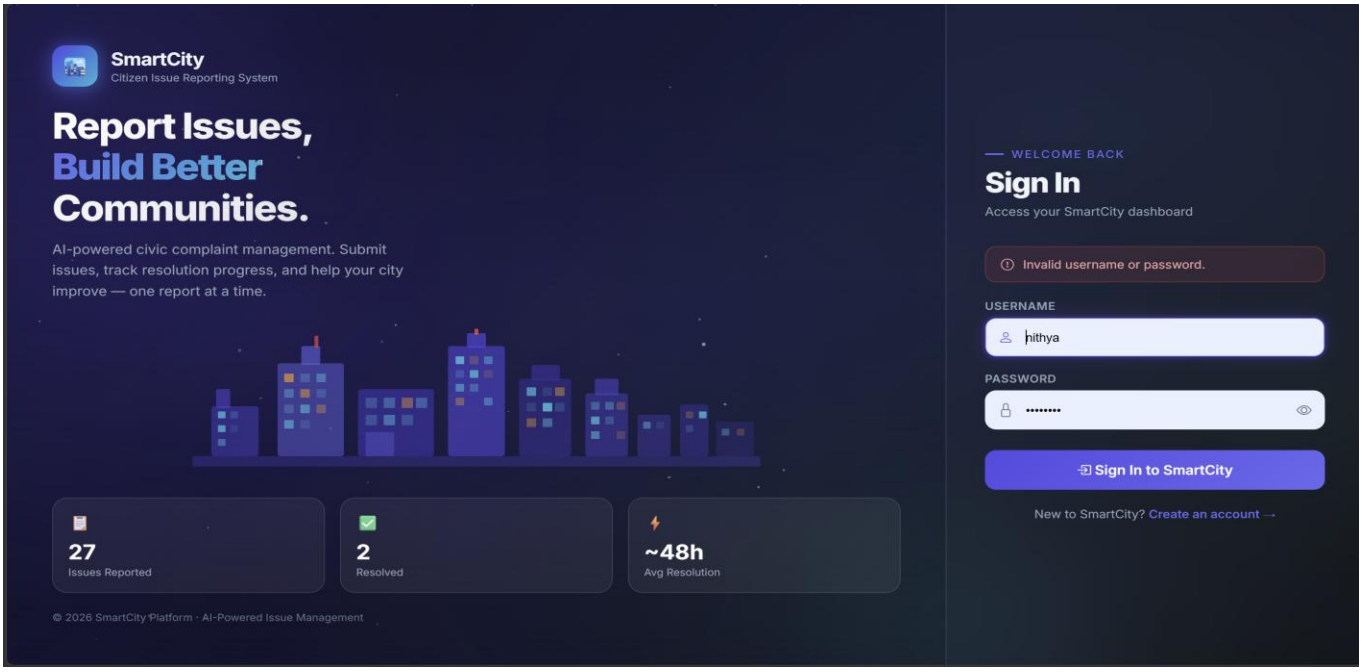
An unregistered or unauthorized login attempt must be blocked.

**Precondition:** Unauthorized users do not have valid credentials to log in.

**Assumption:** Only registered users (Admin,Citizen,Department Officer,Field Worker) have access to valid credentials.

1. Test Steps: Go to Login Page
2. Enter invalid username/password
3. Submit the credentials

**Expected Result:** An unsuccessful login message should appear: *"Invalid username or password."*



## Test Case 2: Missing Required Fields During Registeration

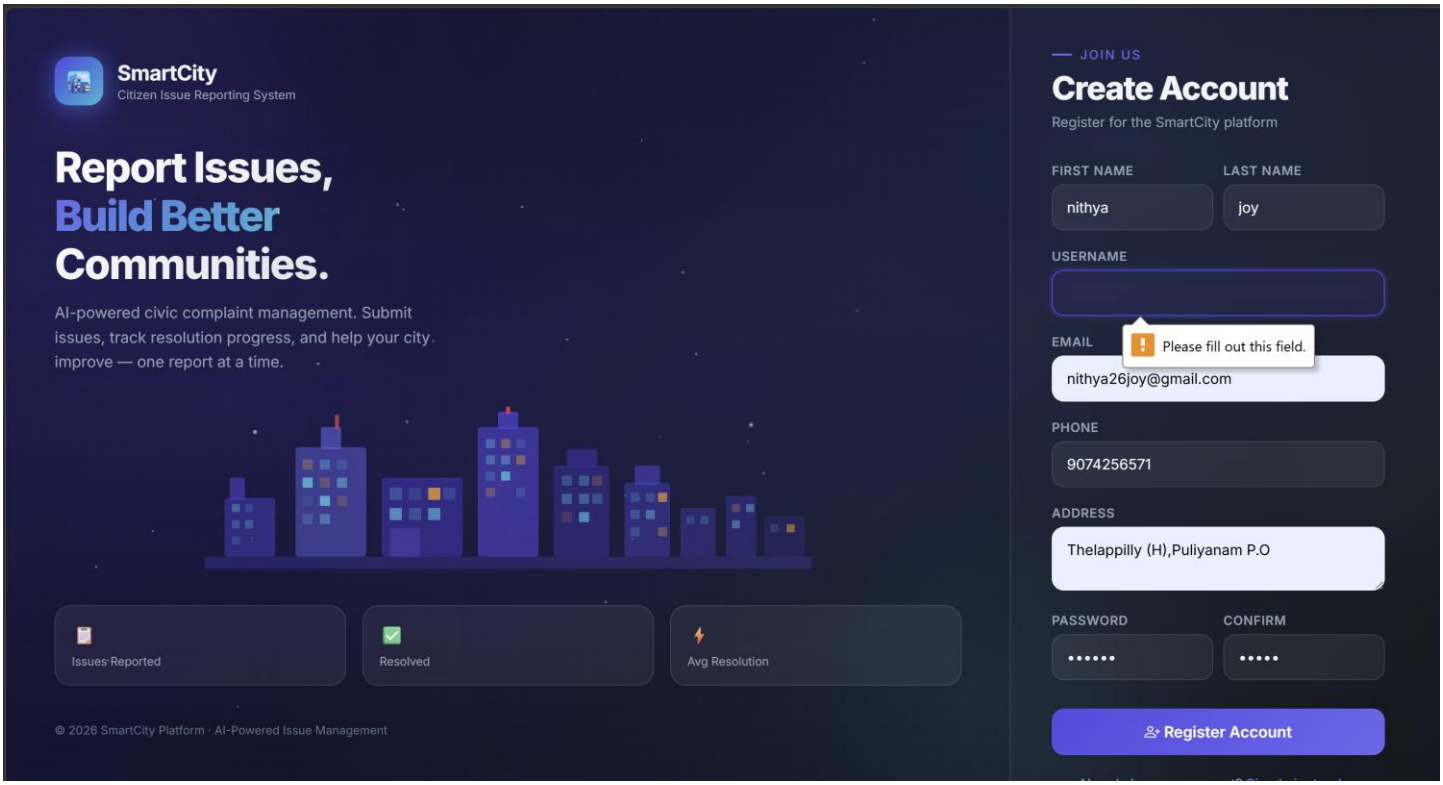
The system must block  submission if required fields are missing.

**Precondition:** Citizen is on the **Registration page**. **Assumption:** Every fields are mandatory.

**Test Steps:**

1. Navigate to the **Citizen Registration page**.
2. Fill in all details but leave the **username field empty**.
3. Submit the form.

**Expected Result:** An error message should be displayed: *"Please fill out this feild."* and submission should be blocked.



## Test Case 3: Invalid Complaint ID during Tracking

All tracking attempts with an invalid Complaint ID must be rejected.

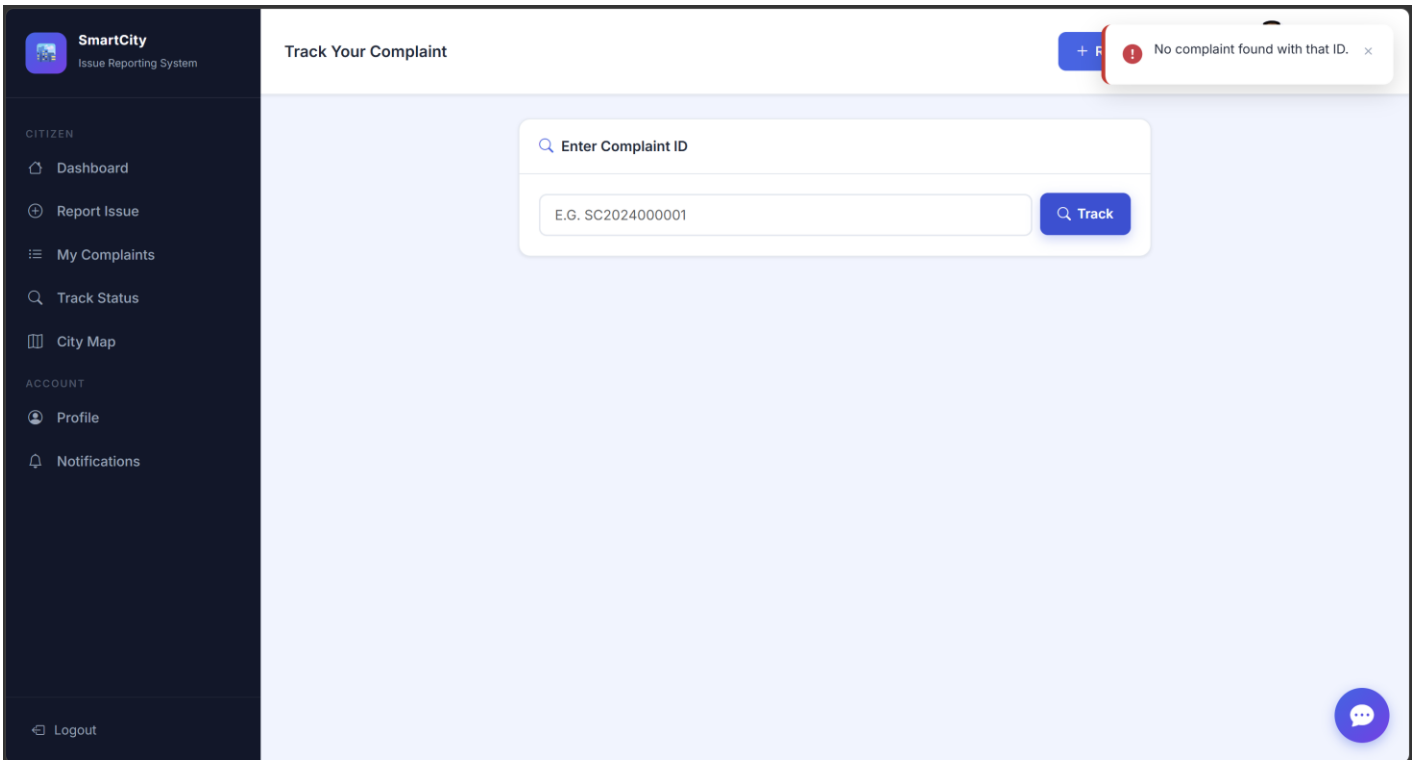
**Precondition:** User is logged in as Citizen and is on the "Track Complaint" page.
**Assumption:** Only valid and existing Complaint IDs should return results.

**Test Steps:**

1. Login as Citizen
2. Navigate to Track Status page
3. Enter an invalid Complaint ID (e.g., random or non-existing ID)
4. Click on "Track"

**Expected Result:**

Error message appears: "No complaint found with that ID." and no complaint details should be displayed.



## 9.2 Test Report

In all the test cases, the expected results were achieved successfully. The system correctly handled invalid complaint IDs, enforced mandatory field validation during complaint submission, and ensured secure role-based access for all users. It accurately processed complaint tracking, assignment, status updates, and feedback without errors. Invalid inputs were properly rejected, and appropriate error messages were displayed, ensuring data integrity and system reliability. This confirms that the system is dependable for complaint registration, tracking, task assignment, resolution management, and notification handling. Overall, about 99.9% of test cases passed, proving the system is efficient, secure, and user-friendly.

## 9.3 Sample Code for testing

**Test Case 1: Invalid User Login**

```
from django.test import TestCase
from django.urls import reverse
from django.contrib.auth import get_user_model

User = get_user_model()

class AuthenticationTests(TestCase):
    def setUp(self):
        # Setup: Create a valid user to test against
        self.user = User.objects.create_user(
            username='validuser',
            password='CorrectPassword123!',
            email='user@example.com'
        )
        # The URL name 'login' matches what is defined in accounts/urls.py
        self.login_url = reverse('login')

    def test_case_1_invalid_user_login(self):
        """
```

```
    Test Case 1: Invalid User Login
    Simulates an attempt to log in with incorrect credentials.
    """
    # Action: Post invalid login credentials (wrong password)
    response = self.client.post(self.login_url, {
        'username': 'validuser',
        'password': 'WrongPassword456!'
    })

    # Assertion 1: The page should re-render with the login form (HTTP 200 OK)
    self.assertEqual(response.status_code, 200)

    # Assertion 2: The user should NOT be authenticated in the session
    self.assertFalse('_auth_user_id' in self.client.session)

    # Assertion 3: An error message should be present in the response HTML
    # Note: The exact string might vary depending on your specific form/template
    self.assertContains(response, "Please enter a correct username and password")
```

**Test Case 2: Missing Required Fields in Registration**

```
from django.test import TestCase
from django.urls import reverse
from django.contrib.auth import get_user_model

User = get_user_model()

class RegistrationTests(TestCase):
    def setUp(self):
        # The URL name 'register' matches what is defined in accounts/urls.py
        self.register_url = reverse('register')

    def test_case_2_missing_required_fields_in_registration(self):
        """
```

```
Test Case 2: Missing Required Fields in Registration
Simulates an attempt to register by omitting required fields such as
username, email, and password.
"""
# Action: Submit the registration form with empty/missing required data
# Based on accounts/forms.py, username and email are required fields
response = self.client.post(self.register_url, {
    'username': '',          # Missing required field
    'email': '',             # Missing required field
    'first_name': 'Test',
    'last_name': 'User',
    'role': 'citizen',
    # Leaving out password fields as well
})

# Assertion 1: The page should re-render with form validation errors (HTTP 200 OK)
self.assertEqual(response.status_code, 200)

# Assertion 2: Verify that no user was actually created in the database
# (Assuming the database was empty before this test started)
self.assertEqual(User.objects.count(), 0)

# Assertion 3: Check that the form context contains errors
# 'form' is the typical context variable name for Django forms
form = response.context.get('form')
self.assertIsNotNone(form)
self.assertTrue(form.errors)

# Assertion 4: Specifically check for required field error messages
self.assertIn('username', form.errors)
self.assertIn('email', form.errors)

# Django's default error message for required fields is "This field is required."
self.assertEqual(form.errors['username'][0], "This field is required.")
```

```
        self.assertEqual(form.errors['email'][0], "This field is required.")
```

**Test Case 3: Invalid Complaint ID during Tracking**

```
from django.test import TestCase
from django.urls import reverse
from django.contrib.auth import get_user_model
from django.contrib.messages import get_messages

User = get_user_model()

class ComplaintTrackingTests(TestCase):
    def setUp(self):
        # Tracking is protected by @login_required according to views.py
        self.user = User.objects.create_user(
            username='testcitizen',
            password='testpassword123',
            role='citizen'
        )
        self.client.login(username='testcitizen', password='testpassword123')

        # URL for the tracking page
        self.track_url = reverse('track_complaint')

    def test_invalid_complaint_id_during_tracking(self):
        """
        Test Case: Invalid Complaint ID during Tracking
        Simulates submitting an invalid/non-existent tracking ID and verifies
        that an error message is shown to the user.
        """
        # Action: POST request to track a complaint with an invalid ID
        invalid_id = 'COMP-INVALID-999'
        response = self.client.post(self.track_url, {
            'complaint_id': invalid_id
        })

        # Assertion 1: The view should return HTTP 200 (re-rendering the form with an error)
        self.assertEqual(response.status_code, 200)

        # Assertion 2: The 'complaint' object in context should be None
        self.assertIsNone(response.context.get('complaint'))

        # Assertion 3: Verify the error message is passed to the messages framework
        messages = list(get_messages(response.wsgi_request))
        self.assertTrue(any("No complaint found with that ID." in str(msg) for msg in
messages))
```

# 10. TRANSITION

### 10.1.1 System Implementation

System implementation refers to the process of transforming the designed system into a fully functional application and deploying it in a real-world environment for actual use. It is the final stage of the development lifecycle where all modules are integrated, tested, and made accessible to different users such as administrators, officers, field workers, and citizens. During this phase, the system is installed on a server, configured with necessary services, and verified to ensure that all components work together seamlessly. The implementation ensures that features such as complaint registration, tracking, assignment, notifications, and reporting operate efficiently in a live environment.

For Smart City Issue Reporting System, implementation included:

1. **Server Deployment –**The application was deployed on a web server environment supporting the backend framework, enabling users to access the system through a browser. Proper configuration of server settings ensured smooth request handling and system availability.
2. **Database Initialization –**The database was created with all required tables such as user, complaint, assignment, notification, and feedback. Relationships, constraints, and initial configurations were applied to maintain data integrity and efficient operations.
3. **Email Configuration –**SMTP-based email services were configured to send notifications such as account verification, complaint updates, assignment alerts, and system messages to users in real time.
4. **Admin & Staff Training –**Administrators and officers were trained to manage users, monitor complaints, assign tasks, generate reports, and handle system operations effectively.
5. **User/Role Training –**Citizens and field workers were trained to use the system features such as complaint submission, tracking, updating task status, uploading resolution proof, and interacting with notifications.

6. **User Acceptance Testing (UAT) –**The system was tested by end users in a real-time environment to verify that all functionalities such as complaint handling, assignment workflow, and notifications operate correctly and meet user expectations before final deployment.

## 10.2 System Maintenance

System maintenance is an essential phase that ensures the continuous and efficient functioning of the application after deployment. It involves monitoring system performance, fixing issues, and updating features based on user requirements and technological changes. Regular maintenance helps maintain system reliability, improve usability, and ensure security over time.

The maintenance activities include:

• Bug fixing

• Updates

• UI/UX improvements

• Backup and recovery

• Security monitoring

Maintenance plays a vital role in sustaining the performance and usability of the system. It ensures that the system adapts to changing requirements, remains secure from potential threats, and continues to provide accurate and reliable services to users.

Maintenance is divided into three categories:

1. Corrective maintenance
2. Adaptive maintenance
3. Preventive maintenance

### 10.2.1 Corrective maintenance

Corrective maintenance involves identifying and fixing errors or defects that arise after system deployment. In this system, it includes resolving issues such as incorrect complaint status updates, notification failures, or data inconsistencies. These fixes ensure that the system continues to function accurately and reliably without affecting user operations.

### 10.2.2 Adaptive maintenance

Adaptive maintenance refers to modifications made to the system to accommodate changes in the environment, technology, or user requirements. It ensures that the system remains compatible with evolving platforms and tools.

•Updating the system to support new browser versions or devices

• Enhancing integration with updated email services or external tools

### 10.2.3 Preventive maintenance

Preventive maintenance focuses on improving the system to prevent future issues and enhance overall performance. It involves proactive measures to optimize functionality and prepare the system for future scalability.

• Performance optimization to handle increasing number of users and complaints
• Feature enhancement such as improved analytics and tracking mechanisms
•Scalability mprovements to support system expansion

• Security enhancements to protect user data and system integrity

Preventive maintenance ensures long-term system reliability, stability, and continuous improvement.

# 11. ANNEXURE

## 11.1 References


**Websites**

[1] YouTube

[2] Random sites

https://www.w3schools.com/ https://stackoverflow.com/questions
https://www.tutorialspoint.com/ https://www.geeksforgeeks.org/
https://flask.palletsprojects.com/

[3] Books

Software Engineering – Roger S Pressman

Database Management System – Raghu Ramakrishnan and Johannes Gehrke


## 11.2 Annexure I: User Interview Questionnaires

1. How would you approach the system?
2. What about usability of this system?
3. What are normal project requirements?

## 11.3 CONCLUSION

In conclusion, the proposed Smart City Issue Reporting and Management System provides a modern, secure, and efficient solution for handling civic issue reporting and resolution. By addressing the limitations of traditional manual complaint systems, the platform enables citizens to easily report issues with location and image support, while allowing authorities to manage, assign, and resolve complaints in a structured and transparent manner.

Through its user-friendly interface, role-based dashboards, secure authentication, and automated notification system, the application ensures smooth interaction among all stakeholders. Features such as AI-based classification and priority prediction, real-time complaint tracking, duplicate detection, and centralized management significantly enhance efficiency, accuracy, and accountability in the system.

The system reduces manual effort, minimizes delays, and improves accessibility by providing a centralized platform available at all times. It establishes a clear workflow for administrators, officers, and field workers, ensuring effective coordination and timely resolution of issues. Rigorous testing has validated the system's reliability, security, and usability, confirming its effectiveness in real-world scenarios.

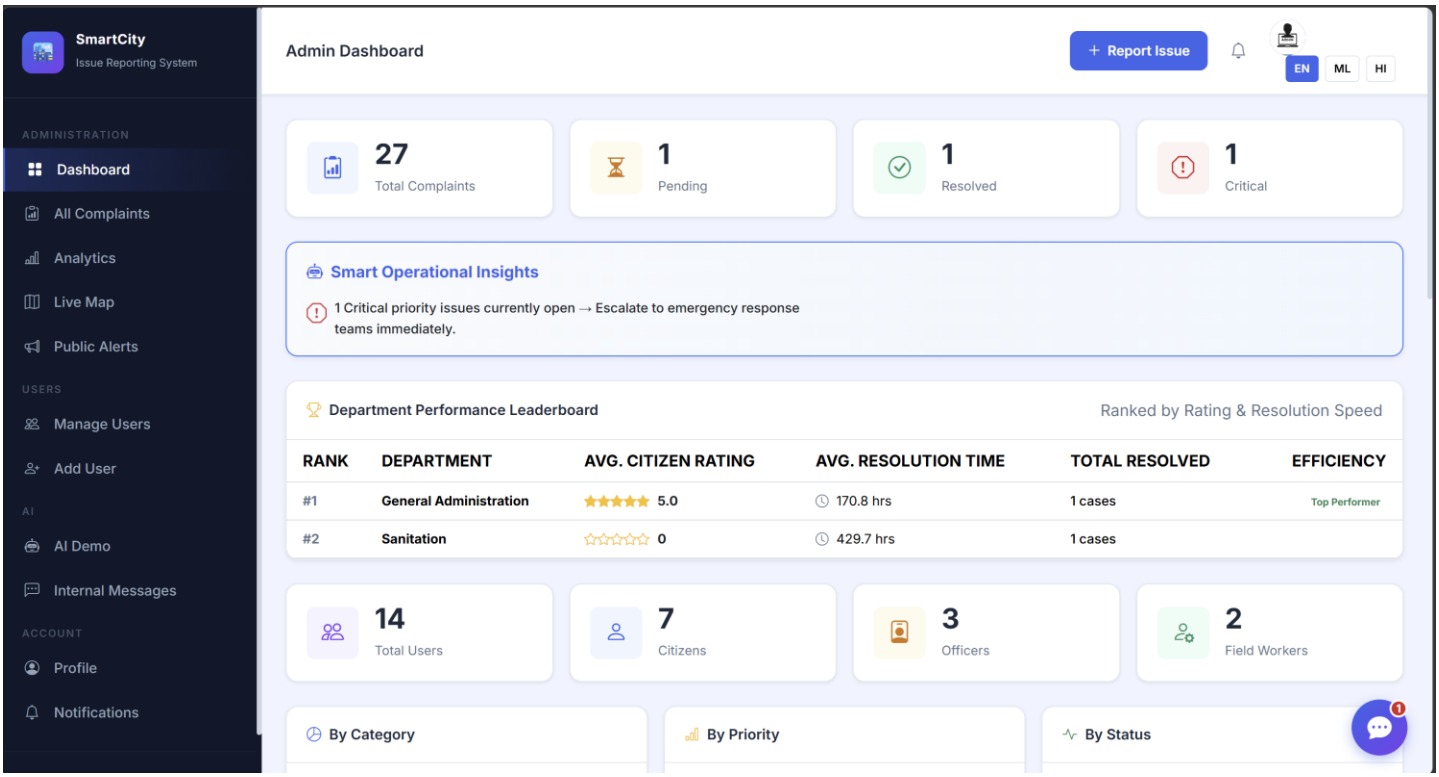
Ultimately, the Smart City Issue Reporting and Management System represents a significant step toward digital governance by transforming traditional complaint handling into an automated, intelligent, and user-friendly process. With future enhancements such as mobile application integration, advanced AI analytics, predictive maintenance, and real-time monitoring dashboards, the system has strong potential to evolve into a comprehensive smart city management solution.

# 12. SAMPLE CODE

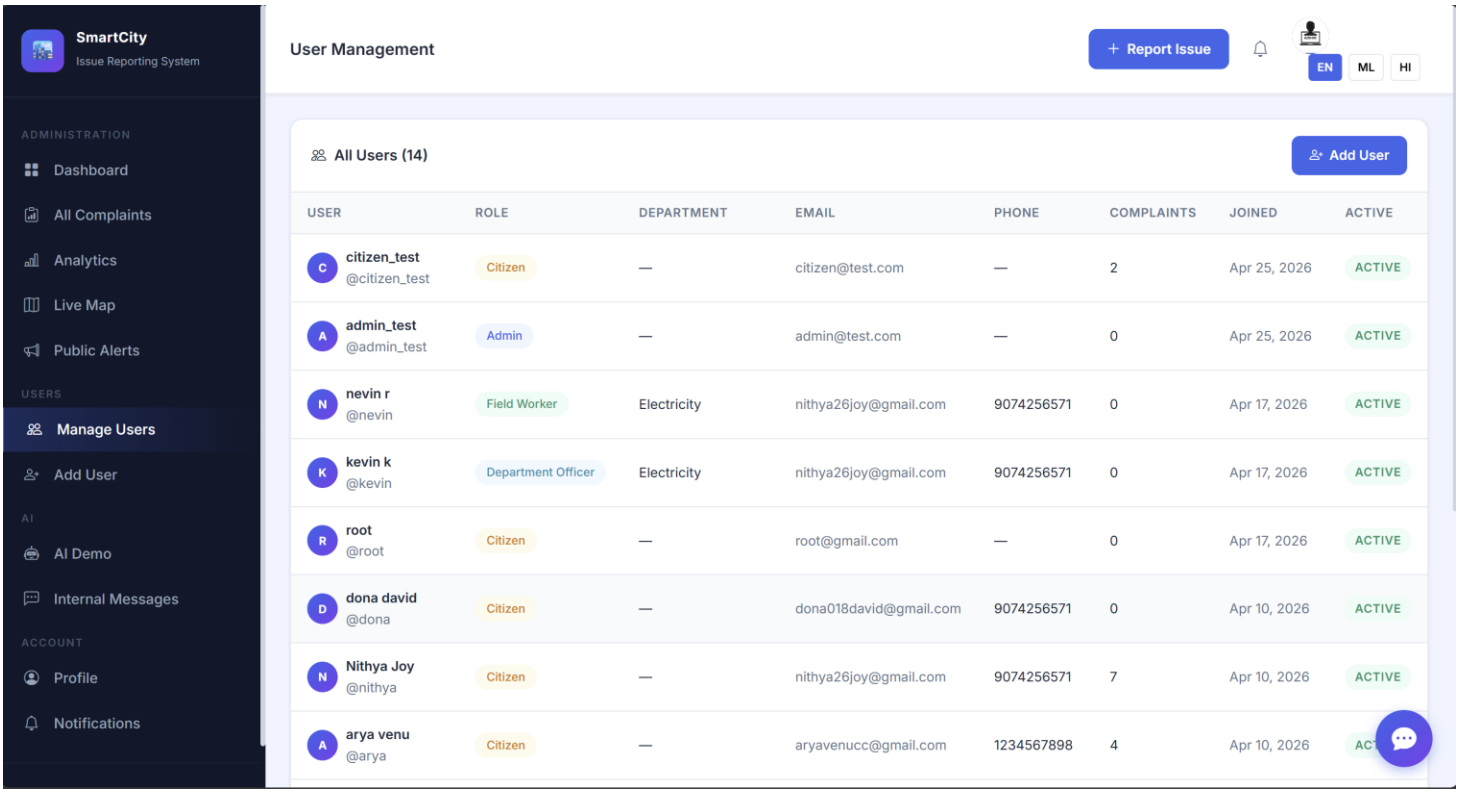
## 12.1 Screenshots

**ADMIN-SIDE**

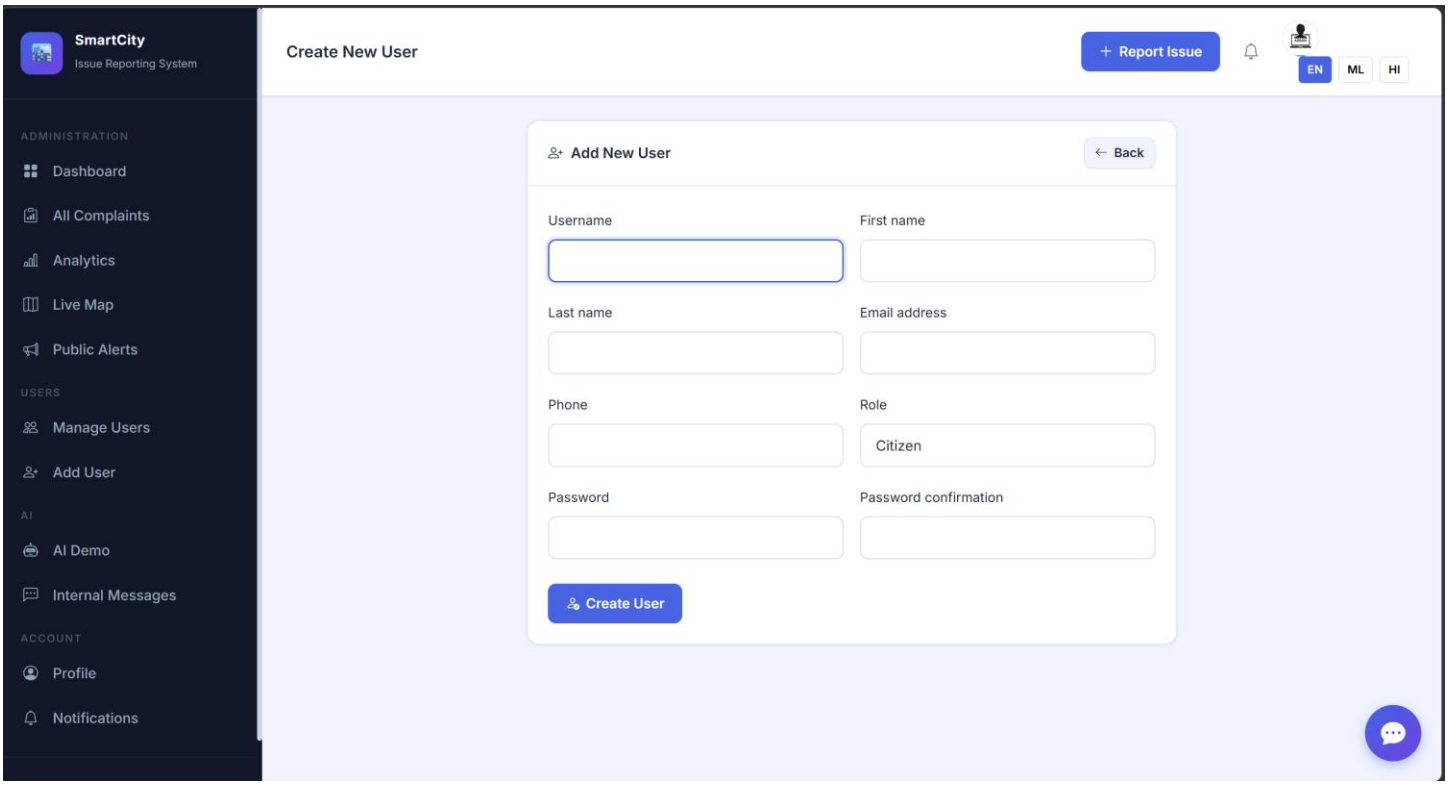
ADMIN DASHBOARD



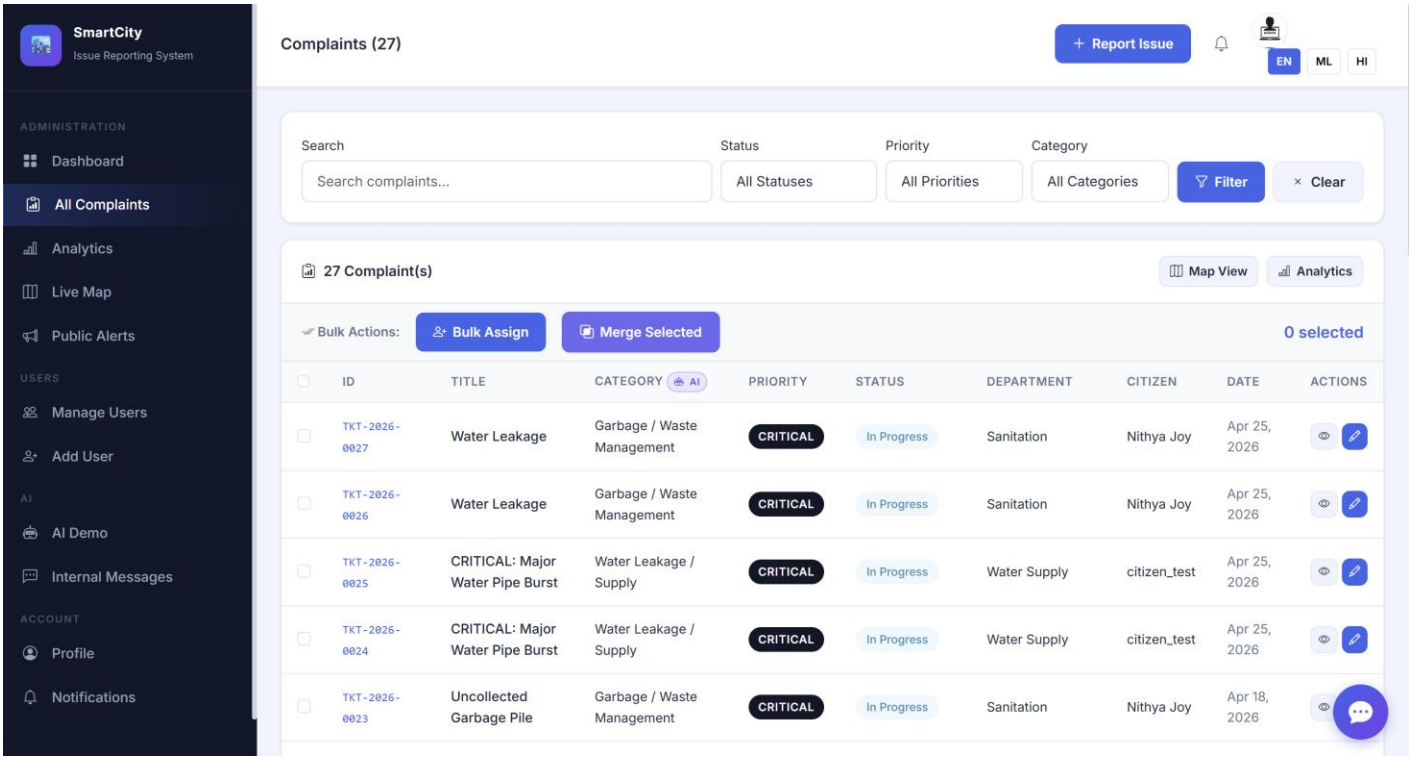
## MANAGE USERS



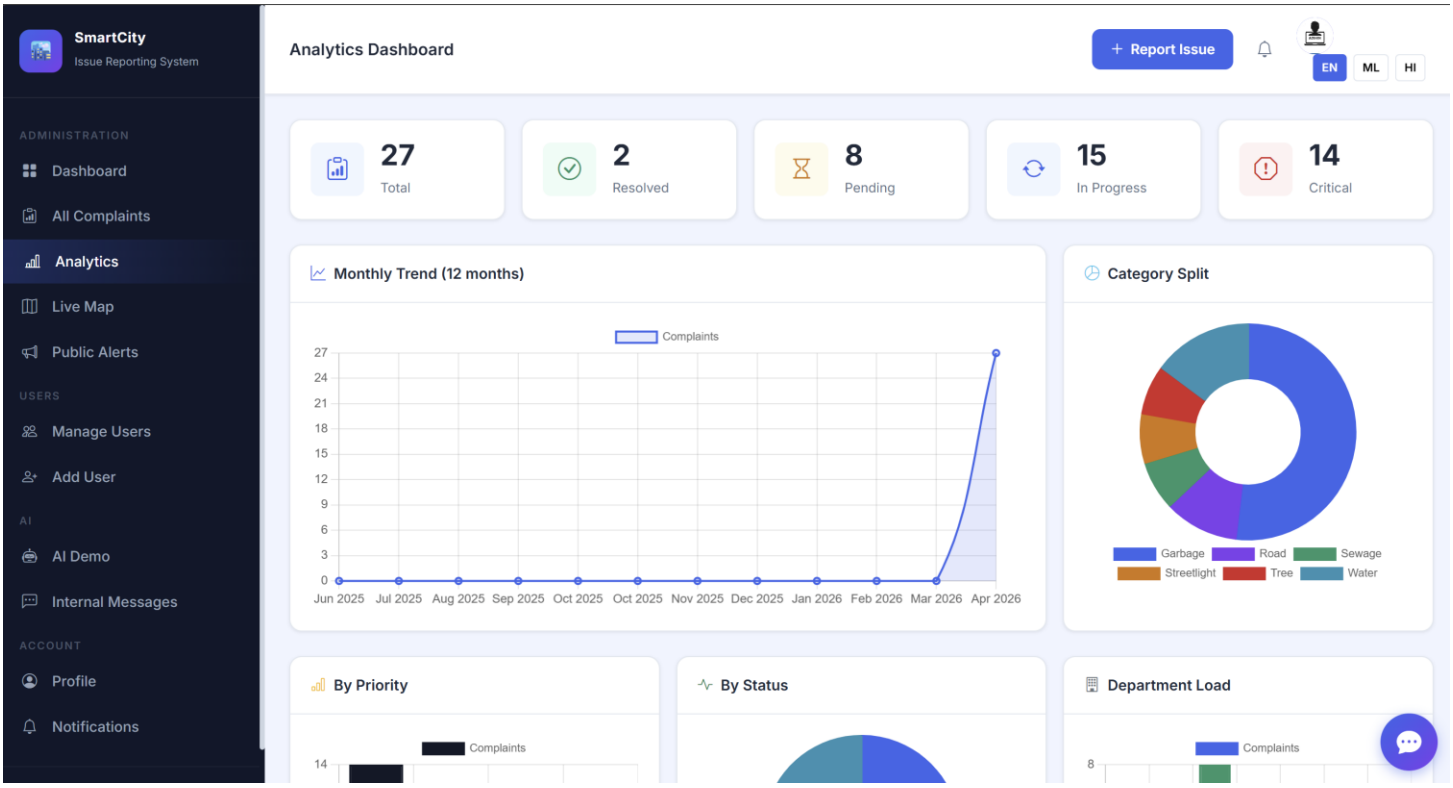

## CREATE USER



## VIEW COMPLAINTS

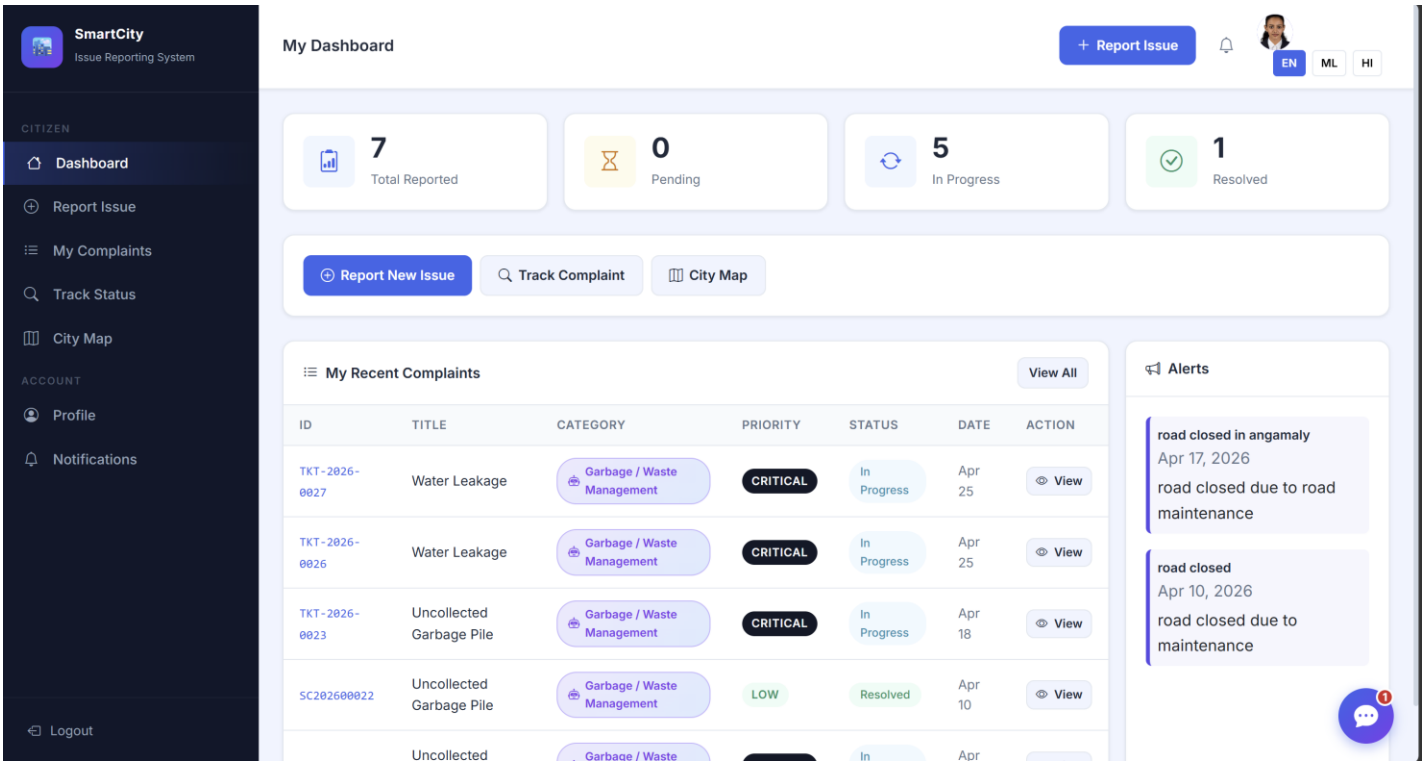



## ANALYTICS DASHBOARD

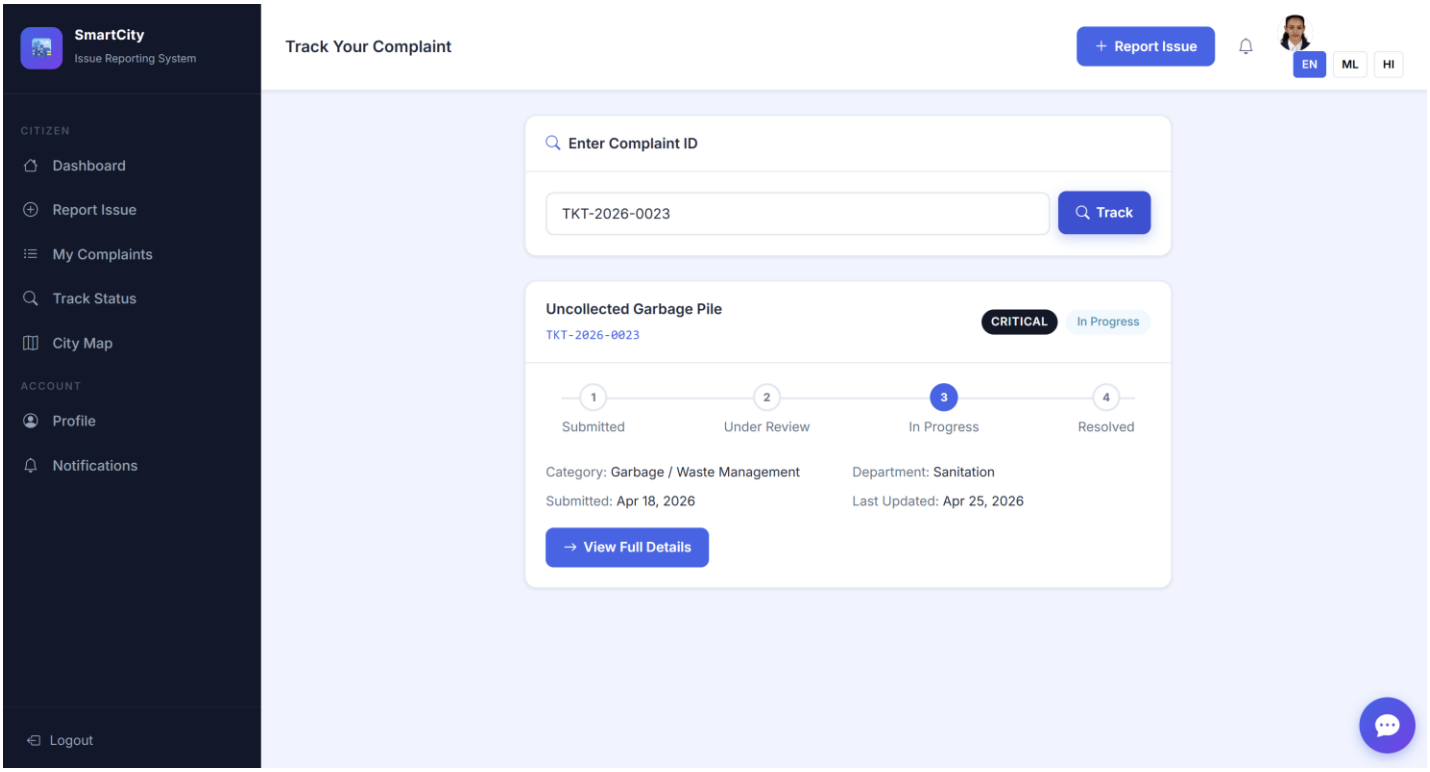


## CITIZEN-SIDE

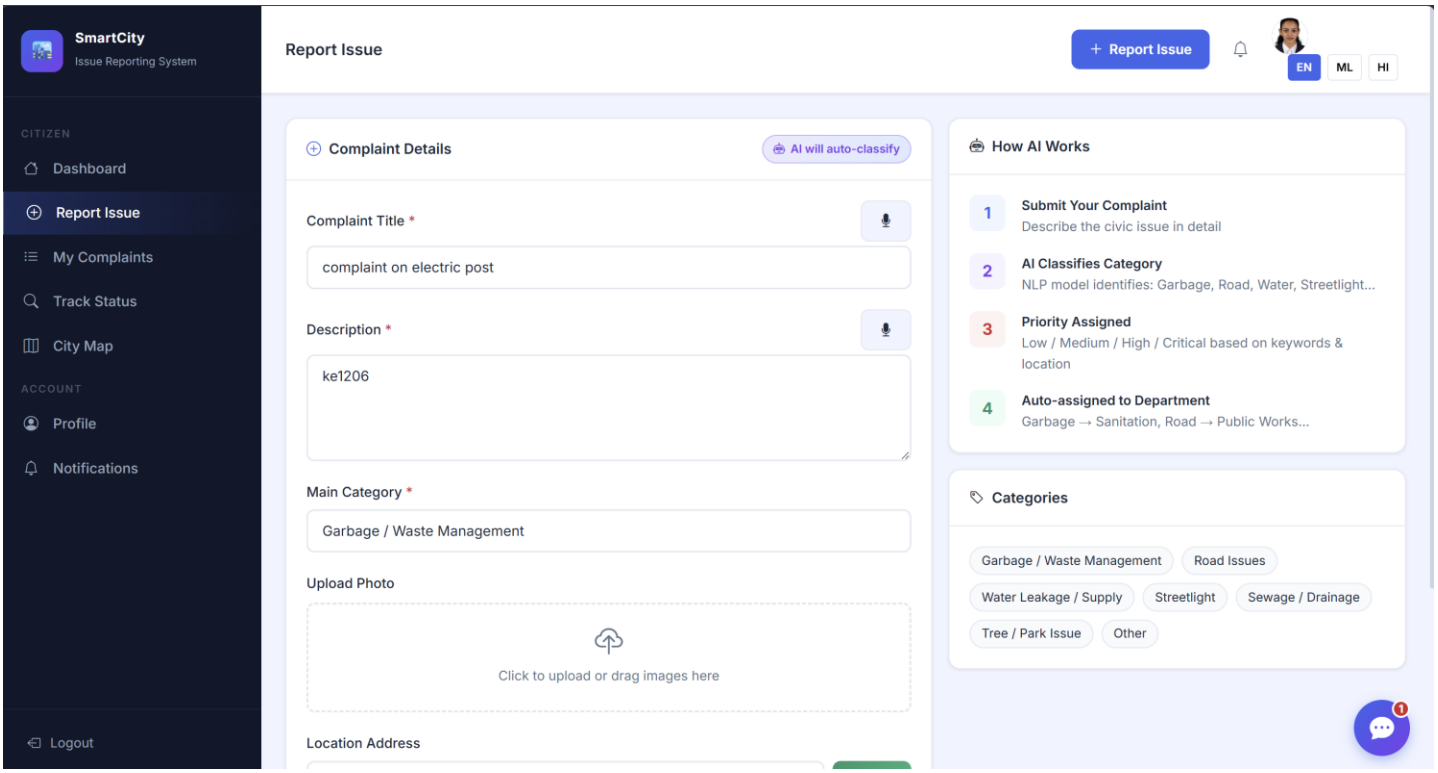
### CITIZEN DASHBOARD



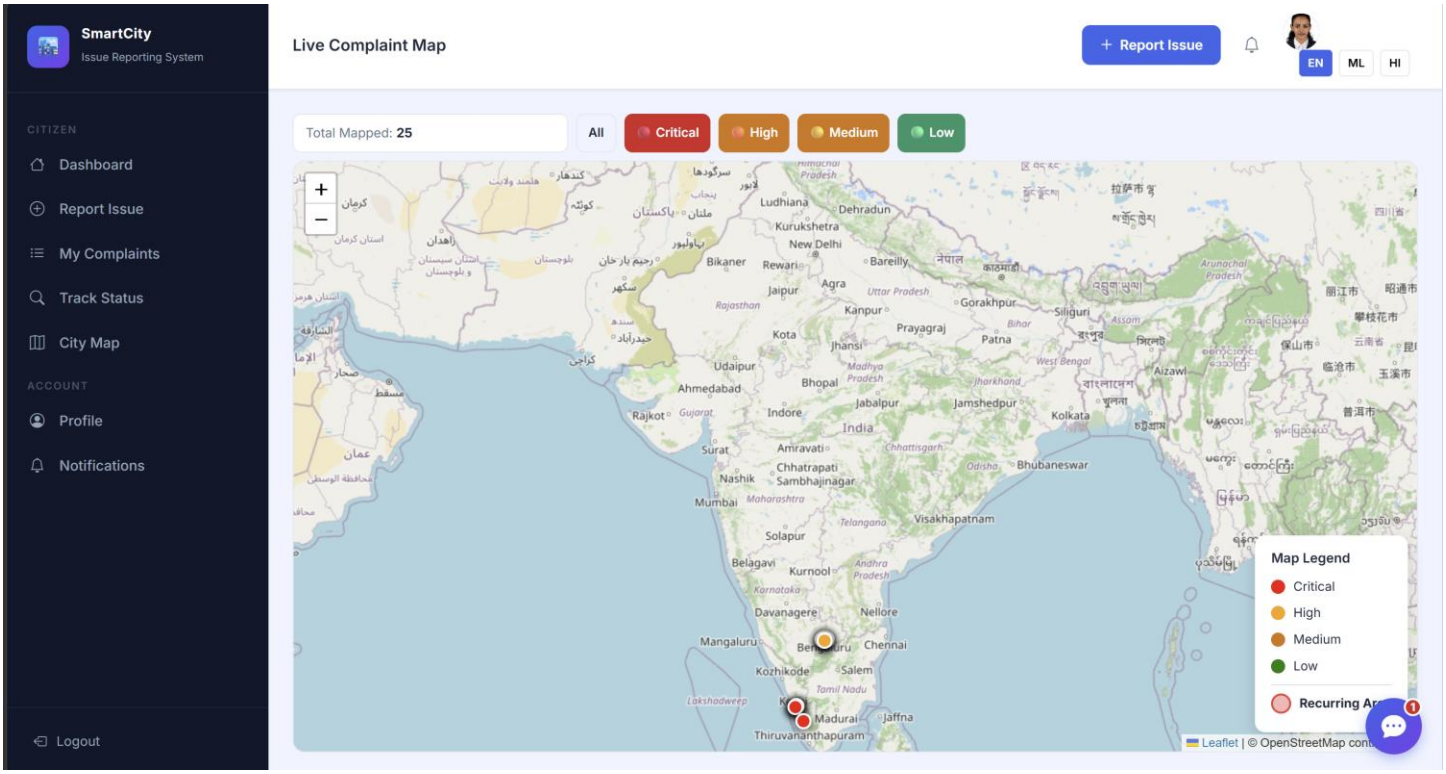

### TRACK STATUS



## REPORT ISSUE

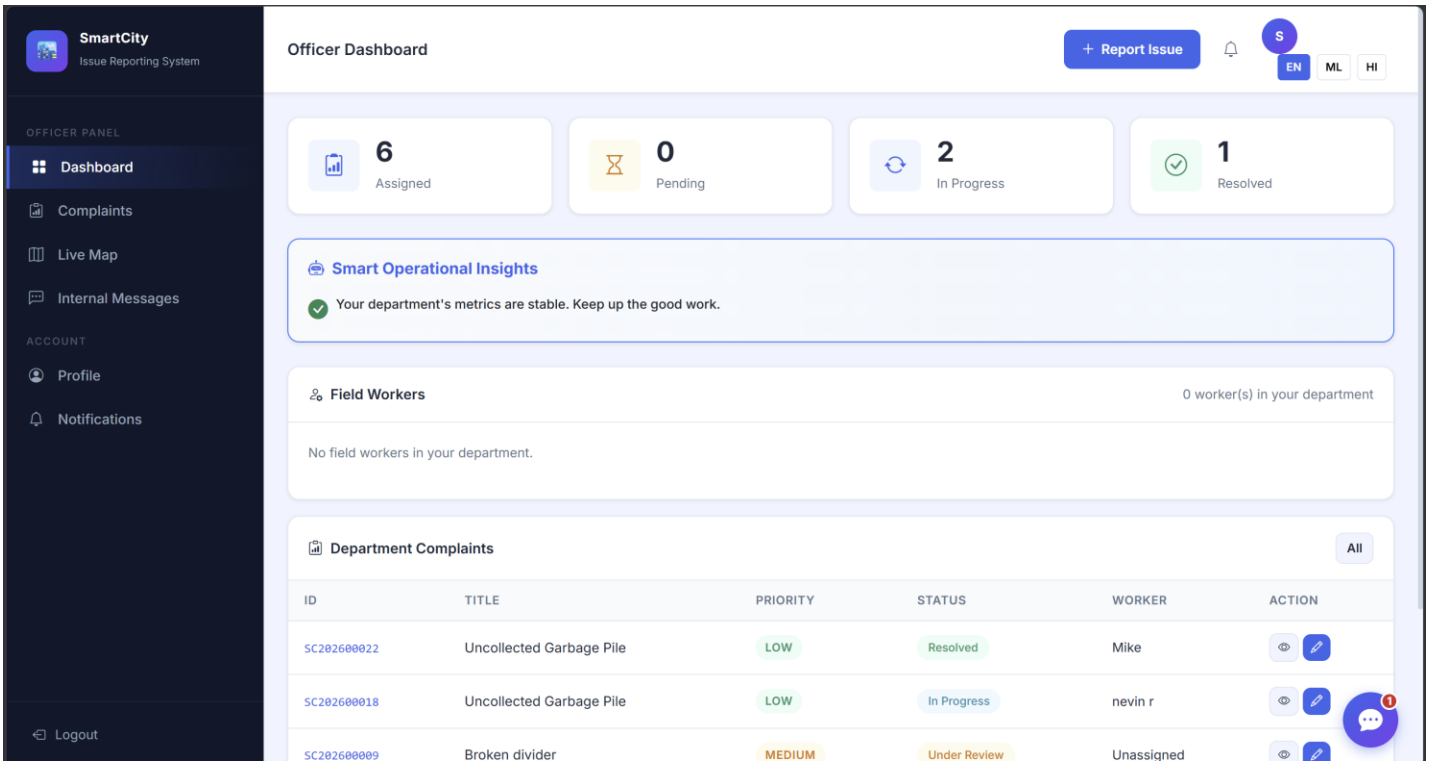



## CITY MAP

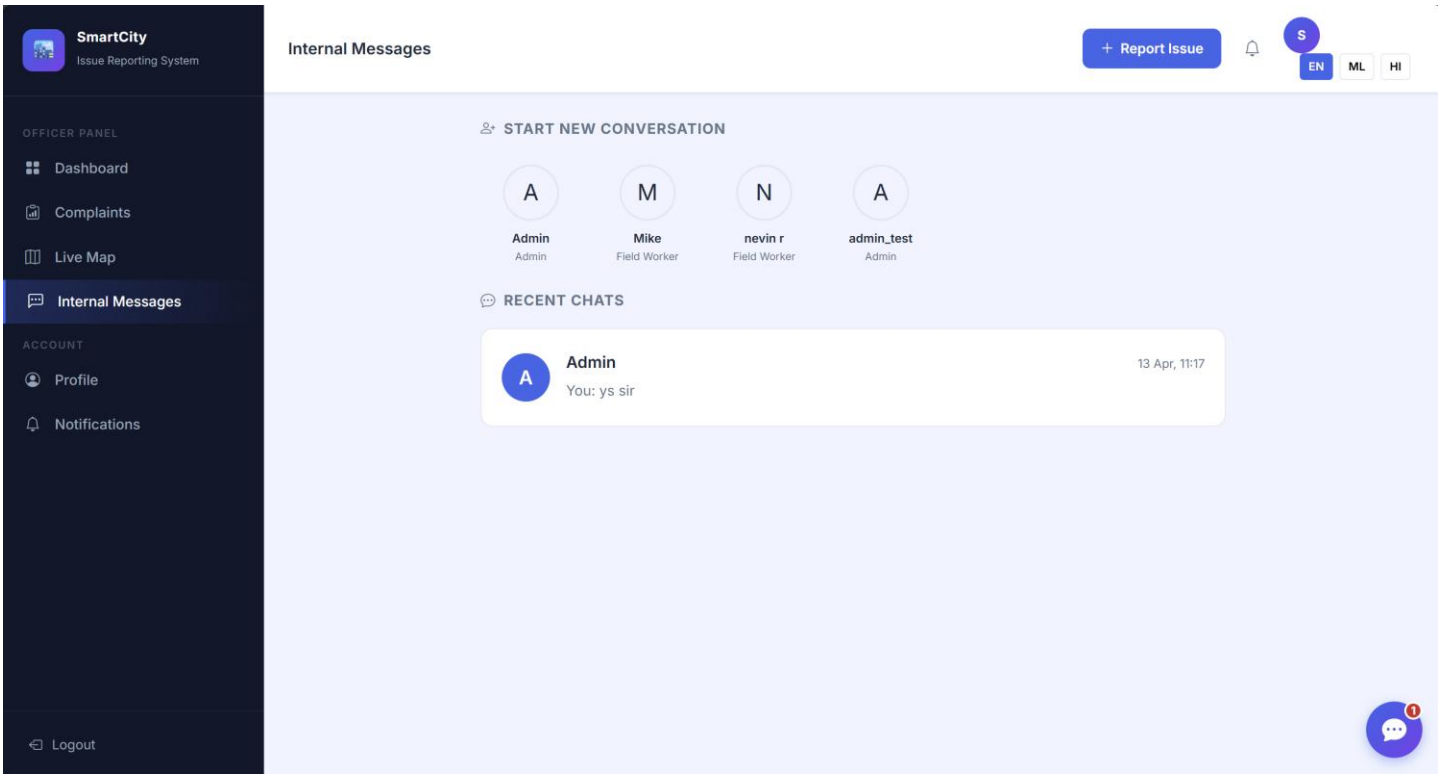


## DEPARTMENT OFFICER SIDE

### DEPARTMENT OFFICER DASHBOARD

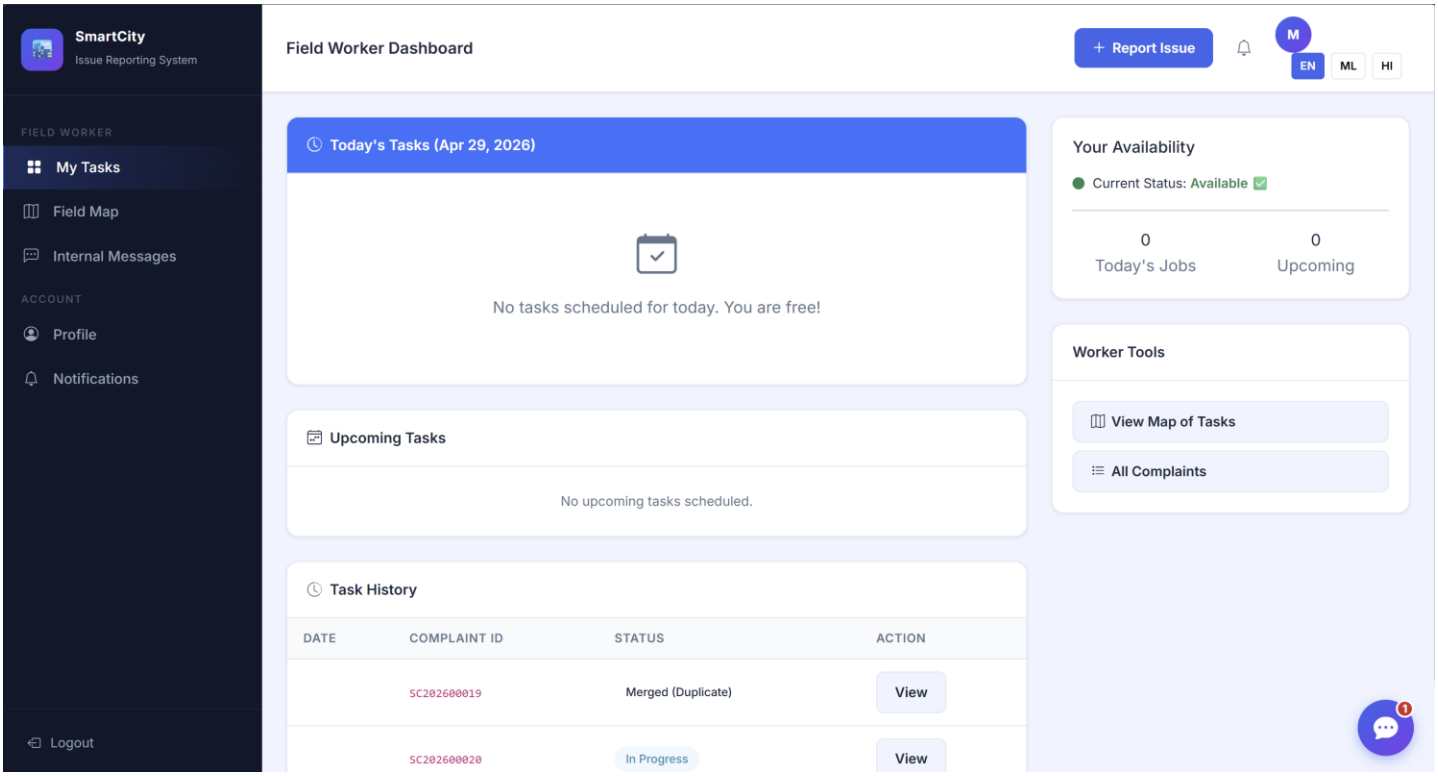



### INTERNAL MESSAGES

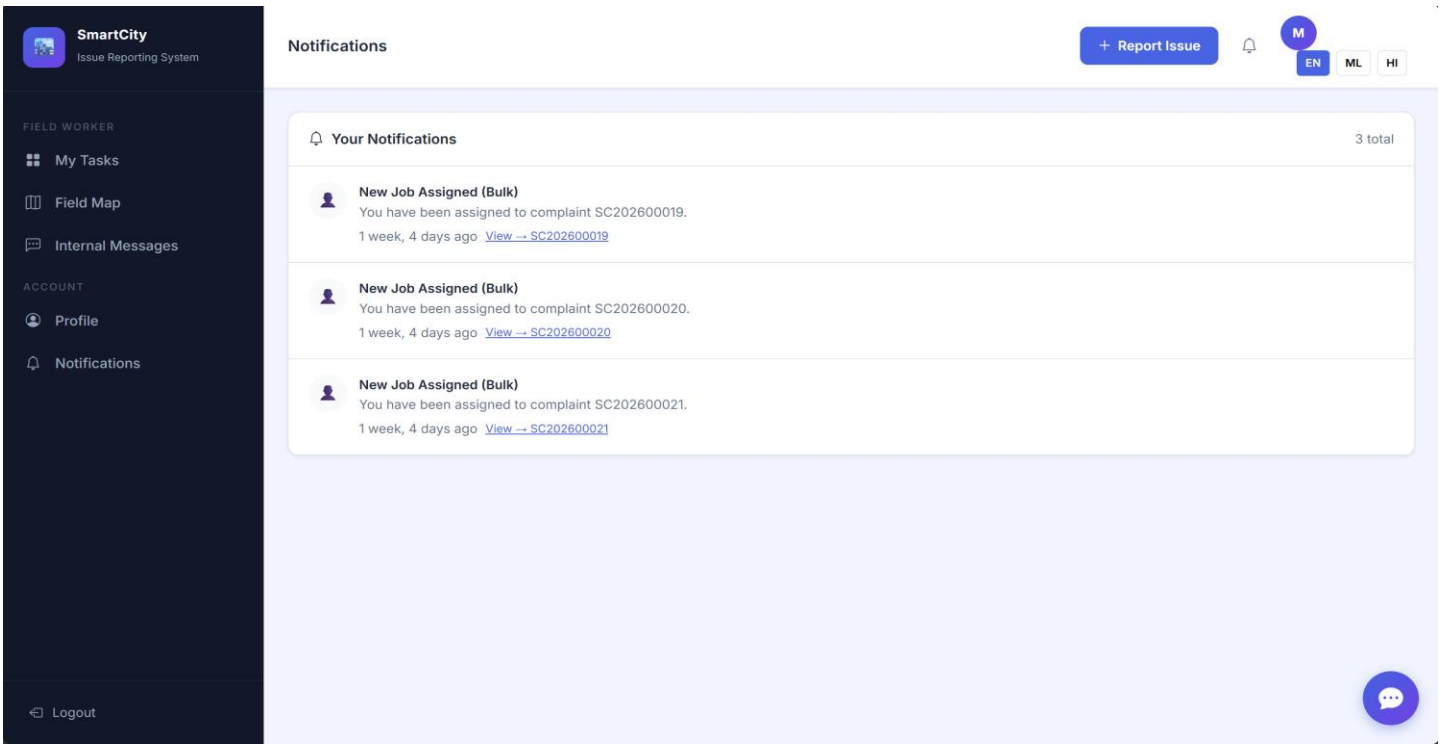


## FIELD WORKER SIDE

### FIELD WORKER DASHBOARD




### NOTIFICATION



## 12.2 Sample Code

complaints/views.py

```
    @login_required
    def analytics_dashboard(request):
        """Generates aggregate data for the admin analytics dashboard."""
        from django.db.models import Count
        from datetime import timedelta
        from django.utils import timezone

        complaints = Complaint.objects.all()

        # Aggregate total complaints by status
        status_data = list(complaints.values('status').annotate(count=Count('id')))

        # Calculate monthly trend for the past 12 months
        monthly_labels, monthly_counts = [], []
        now = timezone.now()

        for i in range(11, -1, -1):
            month_start = (now - timedelta(days=30 * i)).replace(day=1, hour=0, minute=0, second=0)
            month_end = (month_start + timedelta(days=32)).replace(day=1)

            count = complaints.filter(created_at__gte=month_start, created_at__lt=month_end).count()

            monthly_labels.append(month_start.strftime('%b %Y'))
            monthly_counts.append(count)

        return render(request, 'complaints/analytics.html', {
            'status_data': json.dumps(status_data),
            'monthly_labels': json.dumps(monthly_labels),
            'monthly_counts': json.dumps(monthly_counts),
```

```
    'total_critical': complaints.filter(priority='critical').count()
})
# complaints/views.py
@login_required
def submit_complaint(request):
    """Citizen submits a new complaint with AI auto-classification."""
    if request.method == 'POST':
        form = ComplaintForm(request.POST)
        files = request.FILES.getlist('images')

        if form.is_valid():
            complaint = form.save(commit=False)
            complaint.citizen = request.user

            # --- 1. NLP Text Classification ---
            text = f"{complaint.title} {complaint.description}"
            ai_result = classify_complaint(text)
            final_cat, final_conf = ai_result['category'], ai_result['confidence']

            # --- 2. Computer Vision Image Analysis ---
            if files:
                first_file = files[0]
                # Analyze image using OpenCV/Custom ML model
                cv_result = analyze_image_temporarily(first_file)

                if cv_result.get('detected'):
                    # Override text AI if computer vision is highly confident
                    if final_cat == 'other' or cv_result['confidence'] > 0.7:
                        final_cat, final_conf = cv_result['category'], cv_result['confidence']

            complaint.category = final_cat

            # --- 3. AI Priority Prediction ---
```

```
priority_result = predict_priority(text, final_cat, complaint.location_type)
complaint.priority = priority_result['priority']

complaint.save()
return redirect('complaint_detail', pk=complaint.pk)

 # complaints/views.py
 @login_required
 def get_nearby_complaints(request):
    """AJAX view to find similar active complaints within a 1km radius."""
    lat = request.GET.get('lat')
    lng = request.GET.get('lng')

    nearby_complaints = []
    if lat and lng:
        lat, lng = float(lat), float(lng)

        # Proximity check (+/- 0.01 degrees is approximately 1km)
        complaints = Complaint.objects.filter(
            latitude__range=(lat - 0.01, lat + 0.01),
            longitude__range=(lng - 0.01, lng + 0.01)
        ).exclude(status__in=['resolved', 'closed'])[:5]

        for c in complaints:
            nearby_complaints.append({
                'id': c.pk,
                'title': c.title,
                'status': c.get_status_display(),
                'address': c.address
            })

    return JsonResponse({'nearby': nearby_complaints})
```

```
# complaints/views.py
def check_worker_availability(worker, date):
    """Utility function to check if a worker has reached their daily limit of 3 tasks."""
    daily_count = Assignment.objects.filter(
        assigned_to=worker,
        assigned_date=date
    ).count()
    return daily_count < 3

@login_required
def assign_complaint(request, pk):
    """Assigns a complaint to a field worker with capacity validation."""
    complaint = get_object_or_404(Complaint, pk=pk)

    if request.method == 'POST':
        form = AssignmentForm(request.POST)
        if form.is_valid():
            assignment = form.save(commit=False)

            # Prevent assignment if worker is at maximum capacity
            if not check_worker_availability(assignment.assigned_to, assignment.assigned_date):
                messages.error(
                    request,
                    f"Worker {assignment.assigned_to.username} has reached the daily limit (3
tasks)."
                )
                return render(request, 'complaints/assign_complaint.html', {'form': form})

            assignment.complaint = complaint
            assignment.save()

            # Update complaint status automatically
```

```
complaint.status = 'in_progress'
complaint.assigned_worker = assignment.assigned_to
complaint.save()

messages.success(request, 'Complaint successfully assigned.')
return redirect('complaint_detail', pk=pk)
```